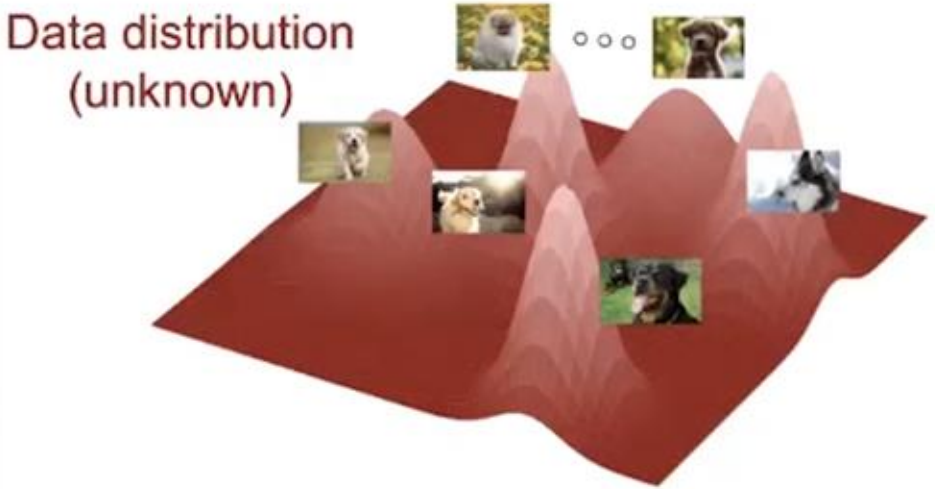
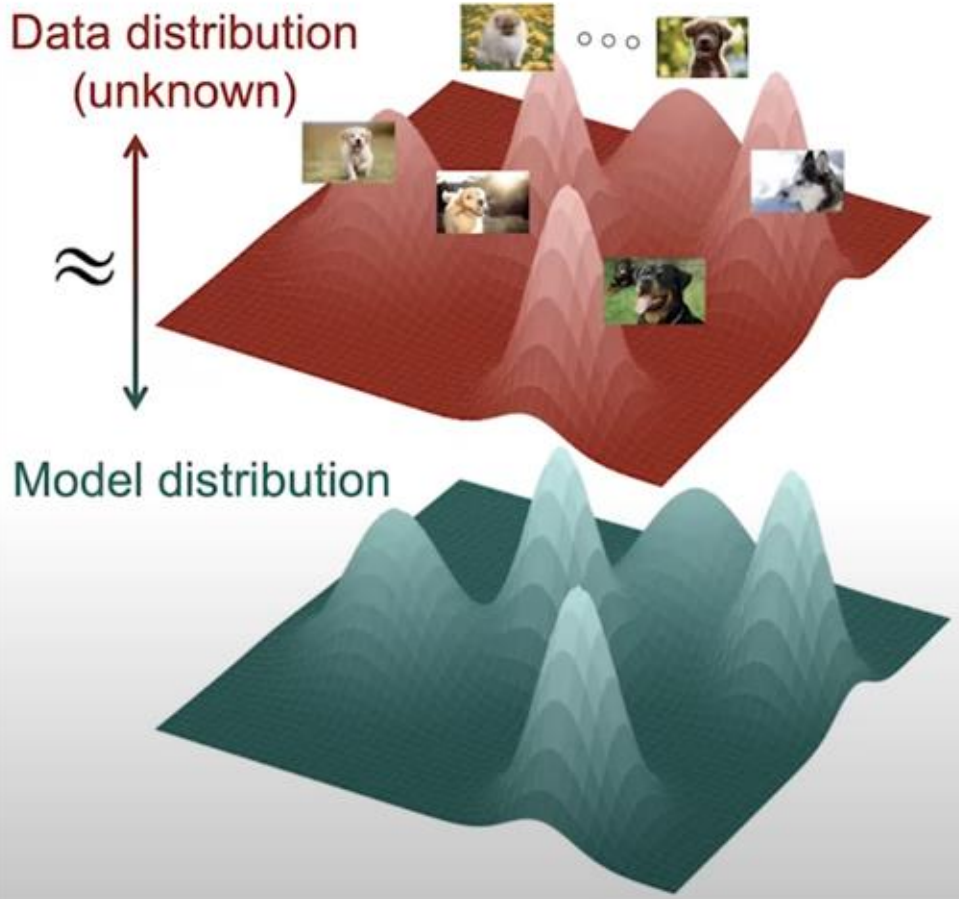
Generative Models

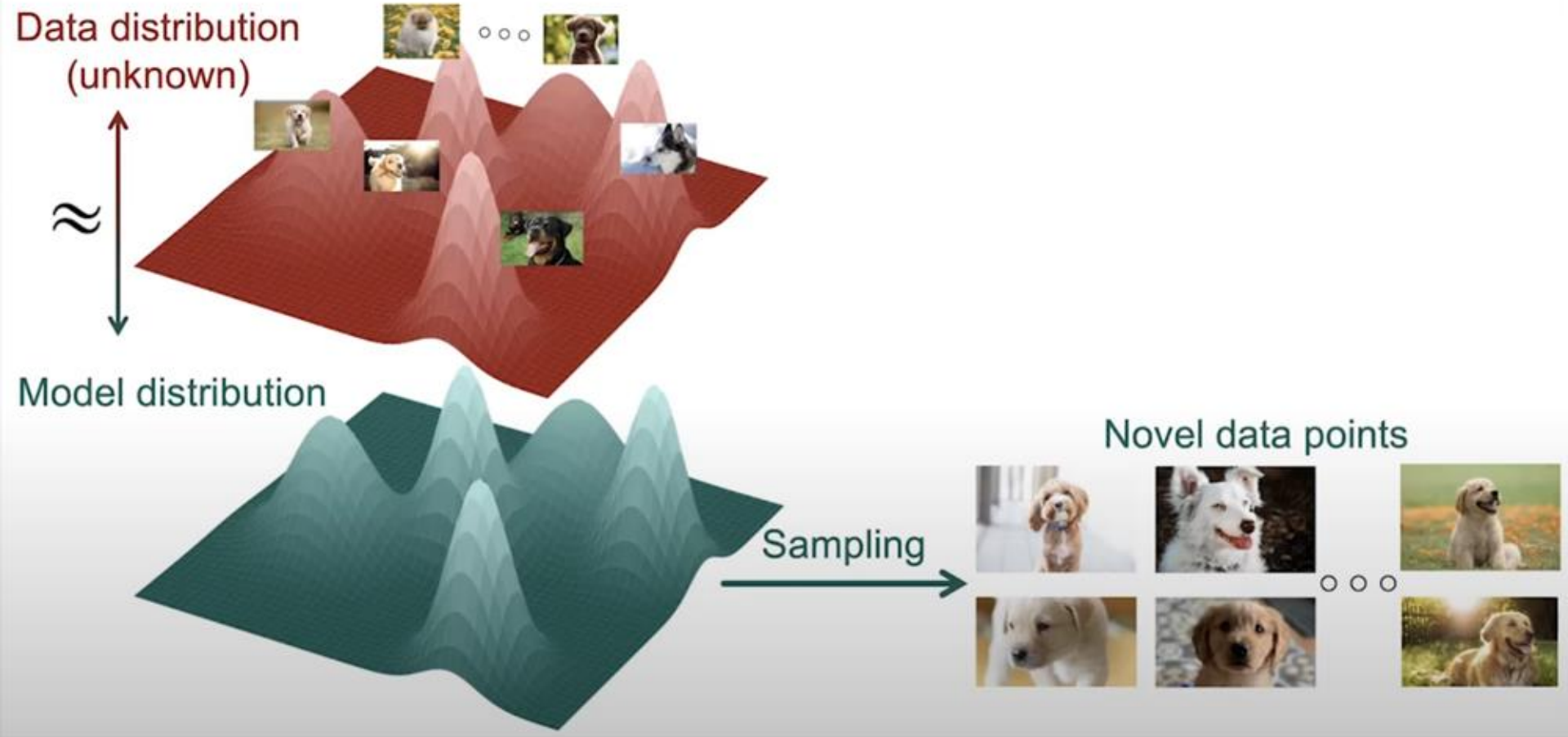
Estimating the probability distribution of data



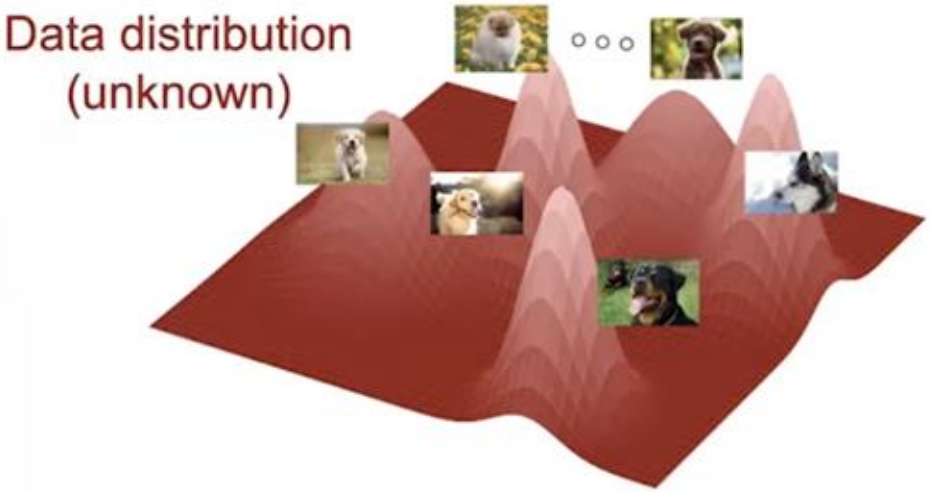
Estimating the probability distribution of data



Estimating the probability distribution of data

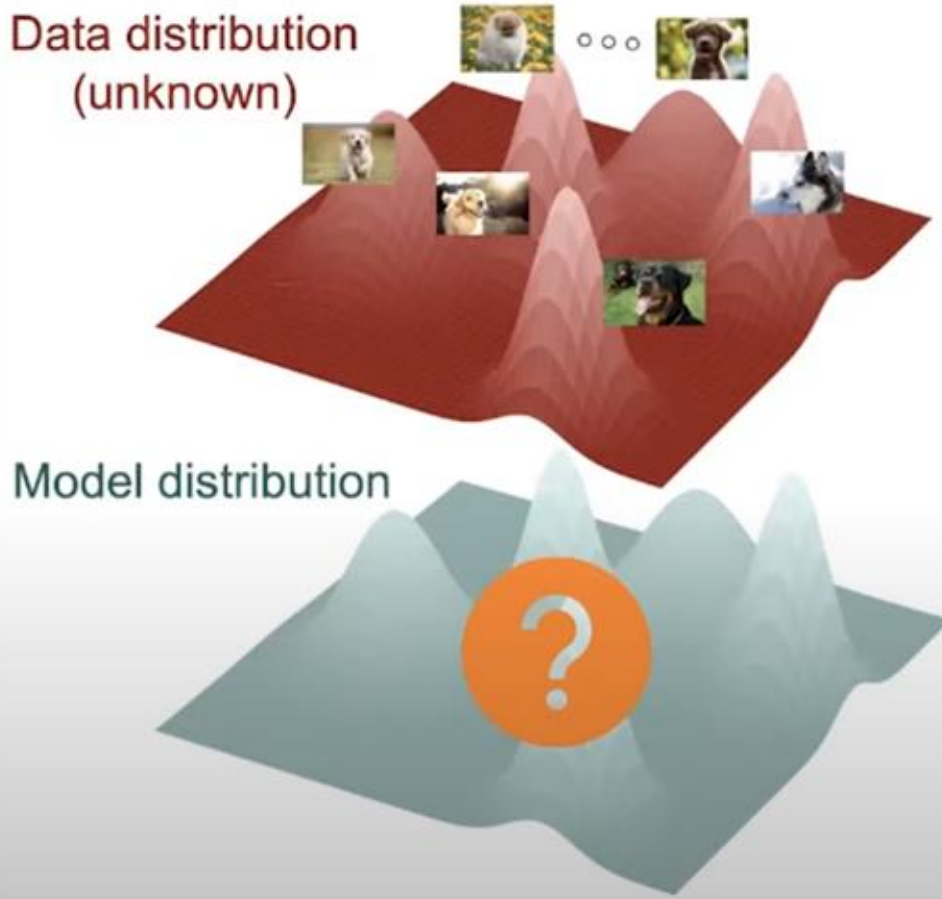


Key challenge



Data distribution is extremely complex for high dimensional data.

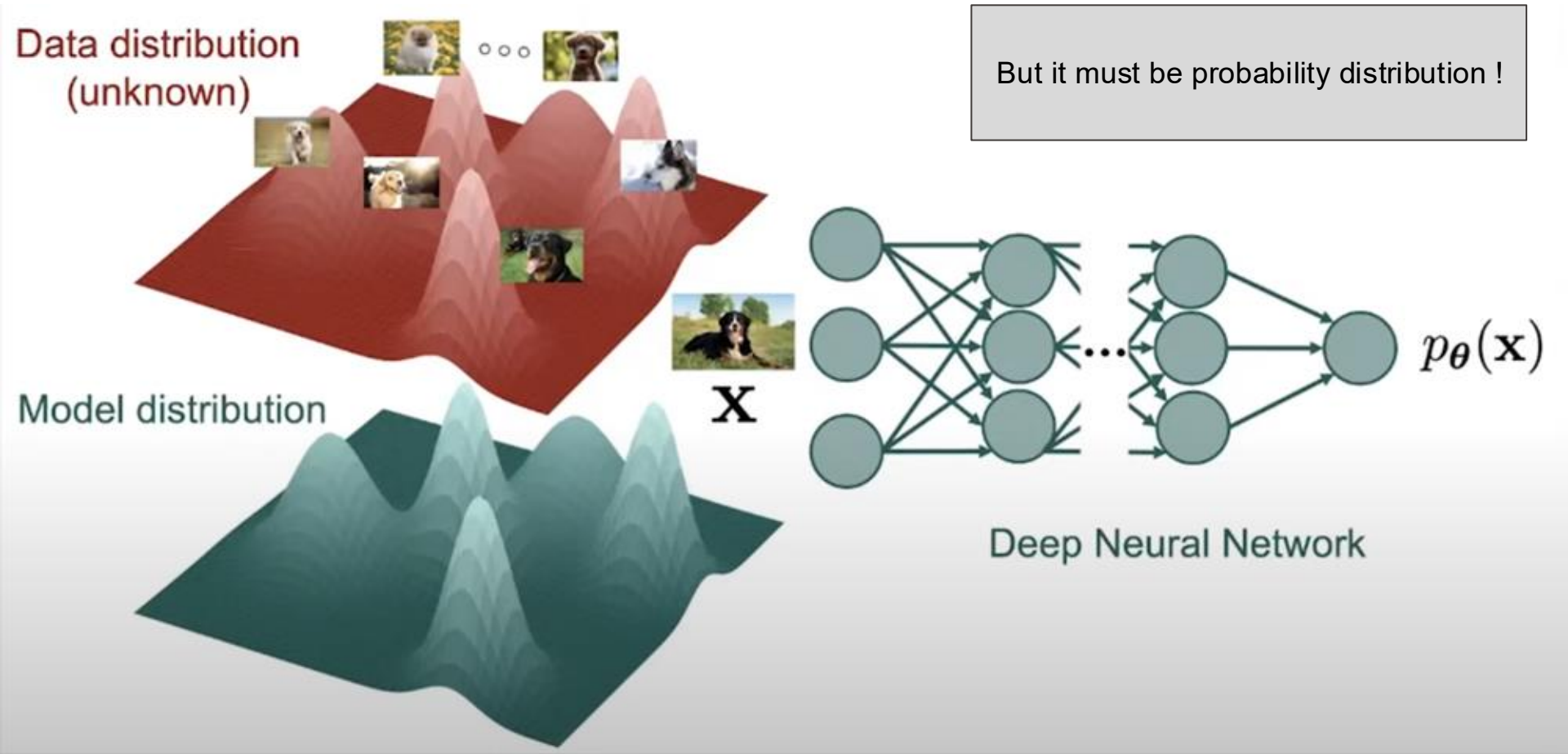
Key challenge



Data distribution is extremely complex for high dimensional data.

How to build a complex model to fit the data distribution?

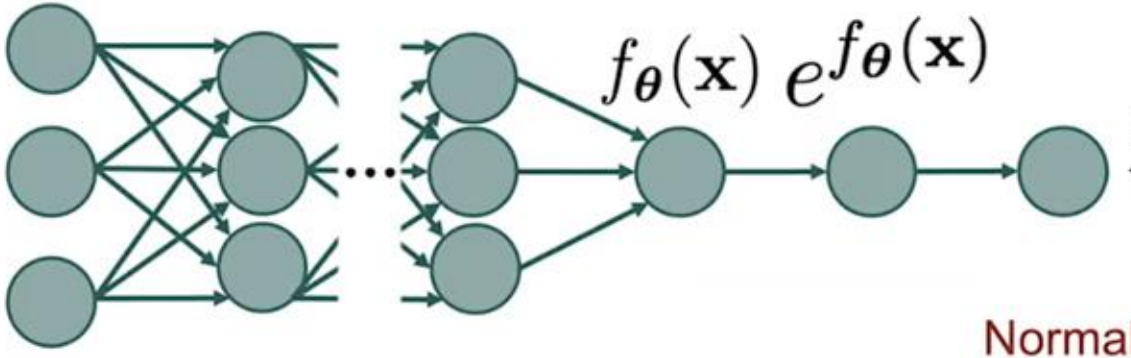
Key challenge



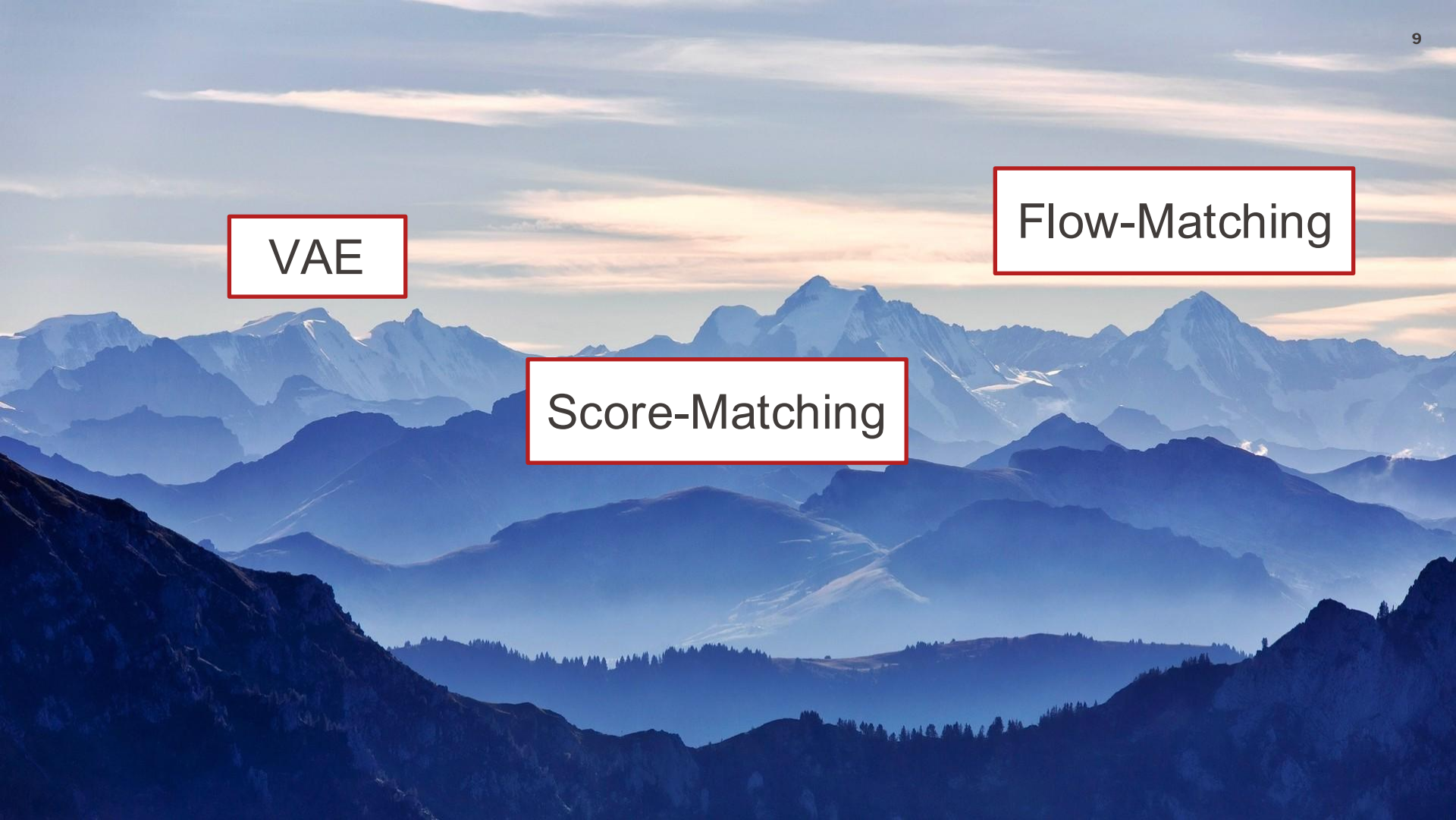
But it must be probability distribution !

Key challenge

$\mathbf{x}$

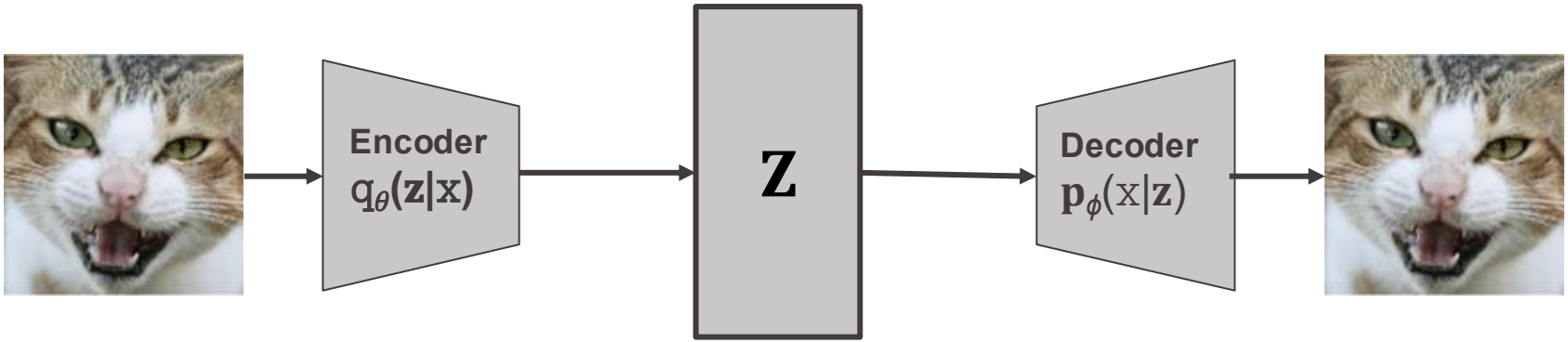


$Z_{\theta} =$?


VAE
Flow-Matching
Score-Matching

Variational AutoEncoder (VAE)

Variational AutoEncoder - Evidence Lower Bound



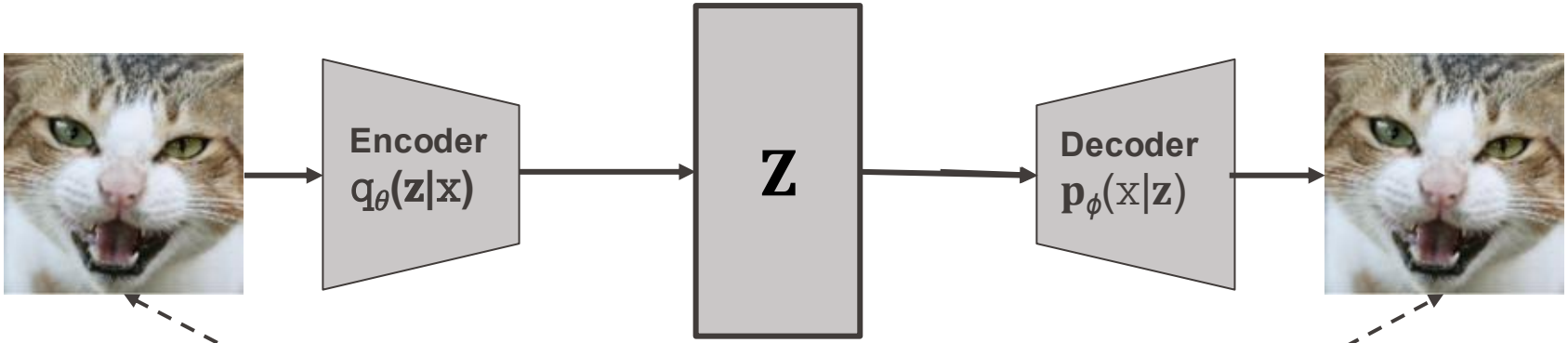
For any data point $\mathbf{x}$, the log-likelihood satisfies:

$$\log p_{\boldsymbol{\phi}}(\mathbf{x}) \geq \mathcal{L}_{\mathrm{ELBO}}(\boldsymbol{\theta}, \boldsymbol{\phi}; \mathbf{x}),$$

where the ELBO is given by:

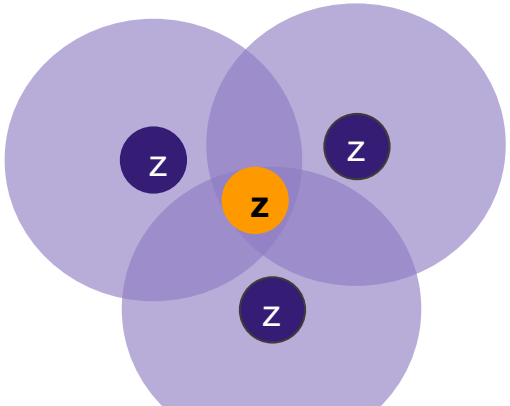
$$\mathcal{L}_{\mathrm{ELBO}} = \underbrace{\mathbb{E}_{\mathbf{z}\sim q_{\boldsymbol{\theta}}(\mathbf{z}|\mathbf{x})}\left[\log p_{\boldsymbol{\phi}}(\mathbf{x}|\mathbf{z})\right]}_{\text{Reconstruction Term}} - \underbrace{\mathcal{D}_{\mathrm{KL}}\left(q_{\boldsymbol{\theta}}(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z})\right)}_{\text{Latent Regularization}}.$$

Variational AutoEncoder

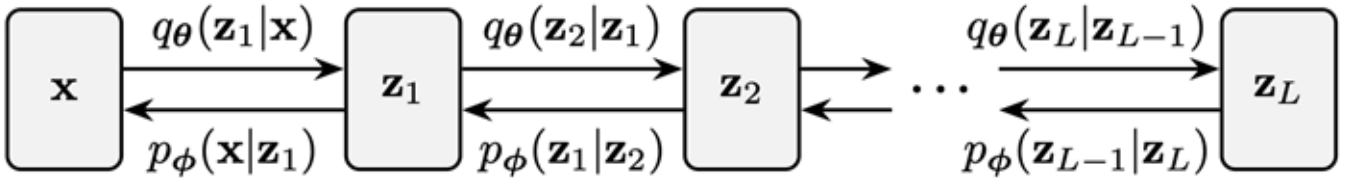


1. Reconstruction Loss

Variational AutoEncoder - Drawback

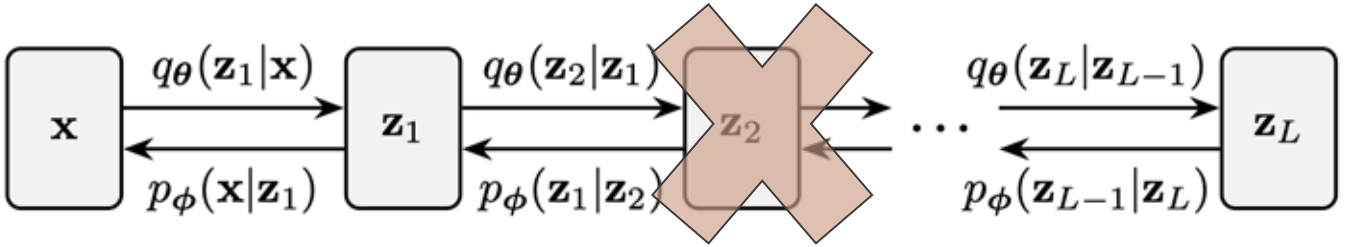


Hierarchical VAE

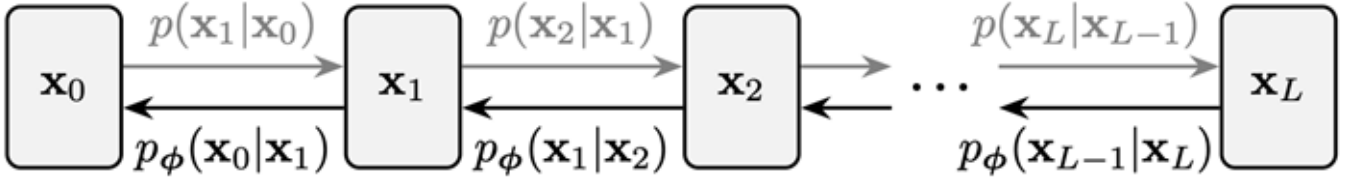


- **Problem**: Their training poses a unique challenge, because encoder and decoder are optimized jointly, learning becomes unstable. Gradient information is often weak in deeper layers making it difficult for them to contribute meaningfully.

Hierarchical VAE → Diffusion Models



Fixed Encoder



Denoising Diffusion Probabilistic Models

[Johnatan Ho et al., NeurIPS 2020]

Data Distribution

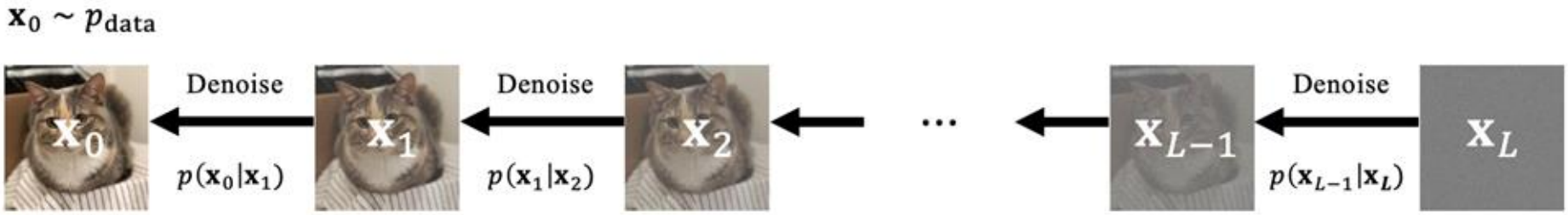
Reverse Diffusion

Noise Distribution

$$p(\mathbf{x}_0) \simeq q(\mathbf{x}_0) \qquad p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$$

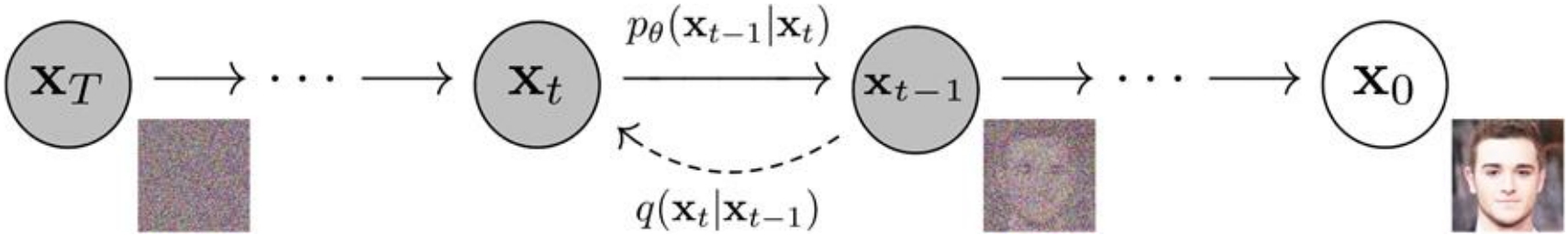
$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) := \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t))$$

Learned drift and covariance functions



Denoising Diffusion Probabilistic Models - Summary

[Johnatan Ho et al., NeurIPS 2020]



Algorithm 1 Training

1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \text{Uniform}(\{1, \dots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on
$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$
6: **until** converged

Algorithm 2 Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \dots, 1$ **do**
3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

Score-based Generative Models

Score function- Definition

$$p(\mathbf{x}) = \frac{\tilde{p}(\mathbf{x})}{Z}, \qquad Z = \int \tilde{p}(\mathbf{x})\, \mathrm{d}\mathbf{x}.$$

While computing Z is intractable, the score depends only on $\tilde{p}$:

$$\nabla_{\mathbf{x}} \log p(\mathbf{x}) = \nabla_{\mathbf{x}} \log \tilde{p}(\mathbf{x}) - \underbrace{\nabla_{\mathbf{x}} \log Z}_{=0} = \nabla_{\mathbf{x}} \log \tilde{p}(\mathbf{x}),$$

It is possible to sample from p only using the score function !

Langevin dynamics sampling

- Sample from $p(\mathbf{x})$ using only the score $\nabla_{\mathbf{x}} \log p(\mathbf{x})$
- Initialize $\mathbf{x}^0 \sim \pi(\mathbf{x})$
- Repeat for $t \leftarrow 1, 2, \cdots, T$

$$\mathbf{z}^t \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$$

$$\mathbf{x}^t \leftarrow \mathbf{x}^{t-1} + \frac{\epsilon}{2} \nabla_{\mathbf{x}} \log p(\mathbf{x}^{t-1}) + \sqrt{\epsilon}\, \mathbf{z}^t$$

Stochasticity

- If $\epsilon \to 0$ and $T \to \infty$, we are guaranteed to have $\mathbf{x}^T \sim p(\mathbf{x})$

- Langevin dynamics + score estimation

$$\boldsymbol{s}_\theta(\mathbf{x}) \approx \nabla_{\mathbf{x}} \log p(\mathbf{x})$$

Working with score functions

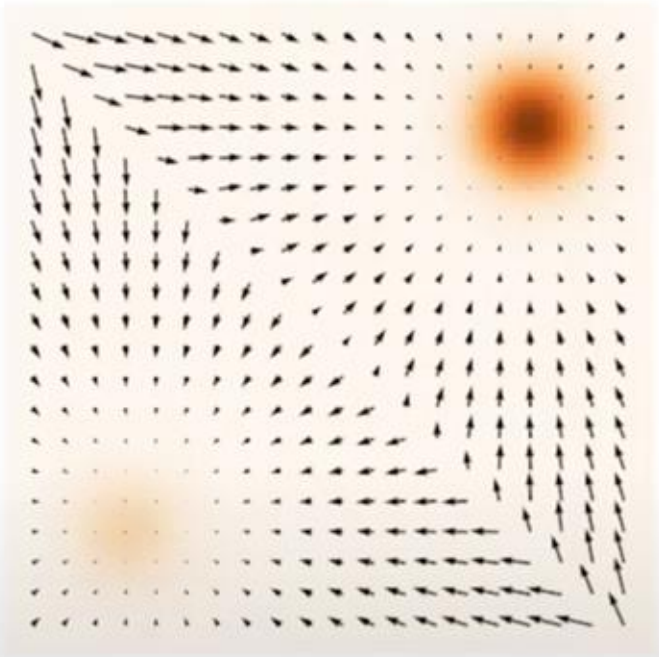
[Yang Song et Stefano Ermon. NeurIPS 2019; Song & Ermon. NeurIPS 2020; Song et al. ICLR 2021; Song et al. ICLR 2022]

$p(\mathbf{x})$

Probability density function

$\nabla_{\mathbf{x}} \log p(\mathbf{x})$

(Stein) score function



Score vs. density function

Score models can be estimated from data

[Yang Song et Stefano Ermon. NeurIPS 2019; Song & Ermon. NeurIPS 2020; Song et al. ICLR 2012; Song et al. ICLR 2022]

Given: $\{\mathbf{x}_1, \mathbf{x}_2, \cdots, \mathbf{x}_N\} \overset{\text{i.i.d.}}{\sim} p_{\text{data}}(\mathbf{x})$

Goal: $\nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x})$

Score Model: $s_\theta(\mathbf{x}) : \mathbb{R}^d \to \mathbb{R}^d \approx \nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x})$

Objective: How to compare two vector fields of scores?

$$\frac{1}{2}\mathbb{E}_{p_{\text{data}}(\mathbf{x})}[\|\nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x}) - s_\theta(\mathbf{x})\|_2^2]$$

(Fisher divergence)

Score models can be estimated from data

[Yang Song et Stefano Ermon. NeurIPS 2019; Song & Ermon. NeurIPS 2020; Song et al. ICLR 2012; Song et al. ICLR 2022]

Given: $\{\mathbf{x}_1, \mathbf{x}_2, \cdots, \mathbf{x}_N\} \overset{\text{i.i.d.}}{\sim} p_{\text{data}}(\mathbf{x})$

Goal: $\nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x})$

Score Model: $s_\theta(\mathbf{x}) : \mathbb{R}^d \to \mathbb{R}^d \approx \nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x})$

Objective: How to compare two vector fields of scores?

$$\frac{1}{2}\mathbb{E}_{p_{\text{data}}(\mathbf{x})}[\| \nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x}) - s_\theta(\mathbf{x}) \|_2^2]$$

(Fisher divergence)

Denoising score matching

- Sample a minibatch of datapoints $\{\mathbf{x}_1, \mathbf{x}_2, \cdots, \mathbf{x}_n\} \sim p_{\text{data}}(\mathbf{x})$
- Sample a minibatch of perturbed datapoints $\{\tilde{\mathbf{x}}_1, \tilde{\mathbf{x}}_2, \cdots, \tilde{\mathbf{x}}_n\} \sim q_\sigma(\tilde{\mathbf{x}})$

$$\tilde{\mathbf{x}}_i \sim q_\sigma(\tilde{\mathbf{x}}_i \mid \mathbf{x}_i)$$

- Estimate the denoising score matching loss with empirical means

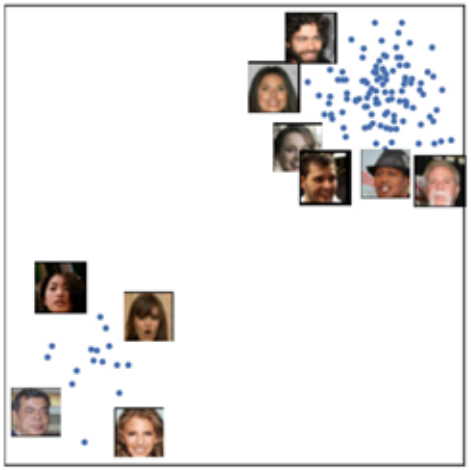
$$\frac{1}{2n}\sum_{i=1}^{n}[\|\boldsymbol{s}_\theta(\tilde{\mathbf{x}}_i) - \nabla_{\tilde{\mathbf{x}}} \log q_\sigma(\tilde{\mathbf{x}}_i \mid \mathbf{x}_i)\|_2^2]$$

- If Gaussian perturbation

$$\frac{1}{2n}\sum_{i=1}^{n}\left[\left\|\boldsymbol{s}_\theta(\tilde{\mathbf{x}}_i) + \frac{\tilde{\mathbf{x}}_i - \mathbf{x}_i}{\sigma^2}\right\|_2^2\right]$$

- Stochastic gradient descent
- Need to choose a very small σ!

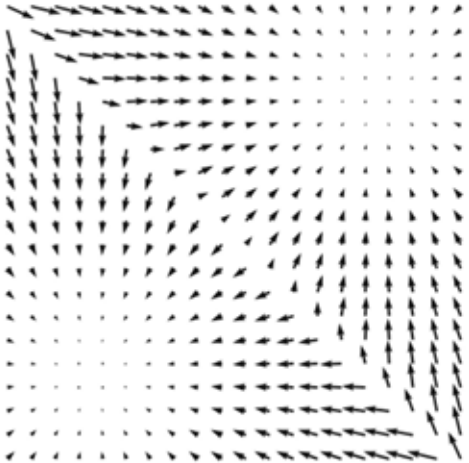
Score-Based Generative Models



Data samples

$\{\mathbf{x}_1, \mathbf{x}_2, \cdots, \mathbf{x}_N\} \overset{\text{i.i.d.}}{\sim} p(\mathbf{x})$

score matching



Scores

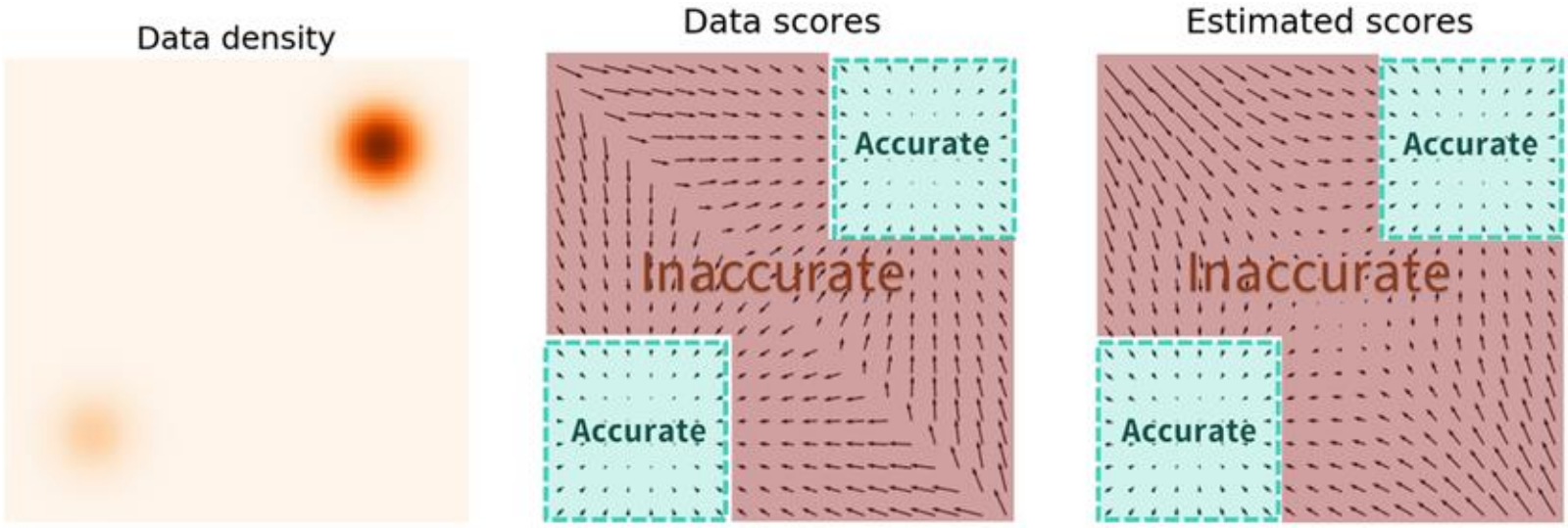
$\mathbf{s}_\theta(\mathbf{x}) \approx \nabla_{\mathbf{x}} \log p(\mathbf{x})$

Langevin dynamics

New samples

Any pitfalls ?

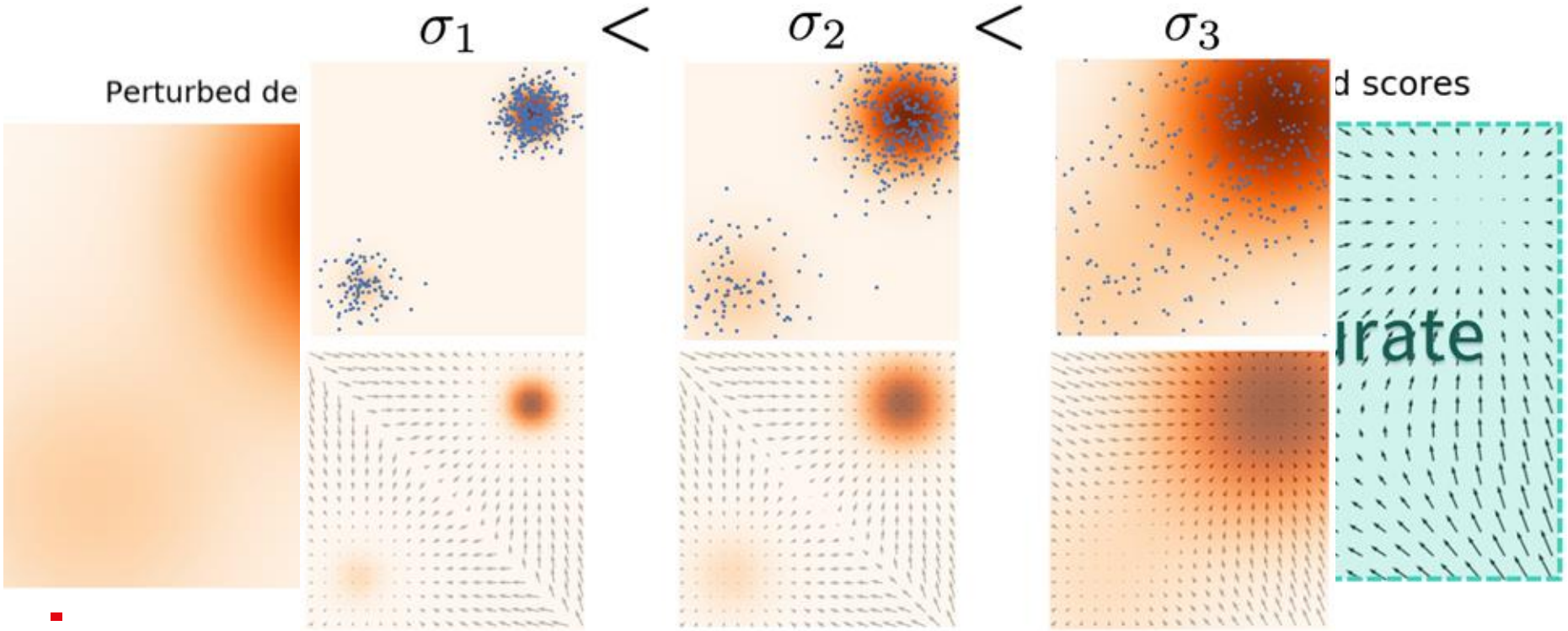
Score-Based Generative Models



Inaccurate Estimation in low-density regions

Score-Based Generative Models - Remedy

A solution is to perturb the data points with different degree of noise and train score-based models on the noisy data points instead.

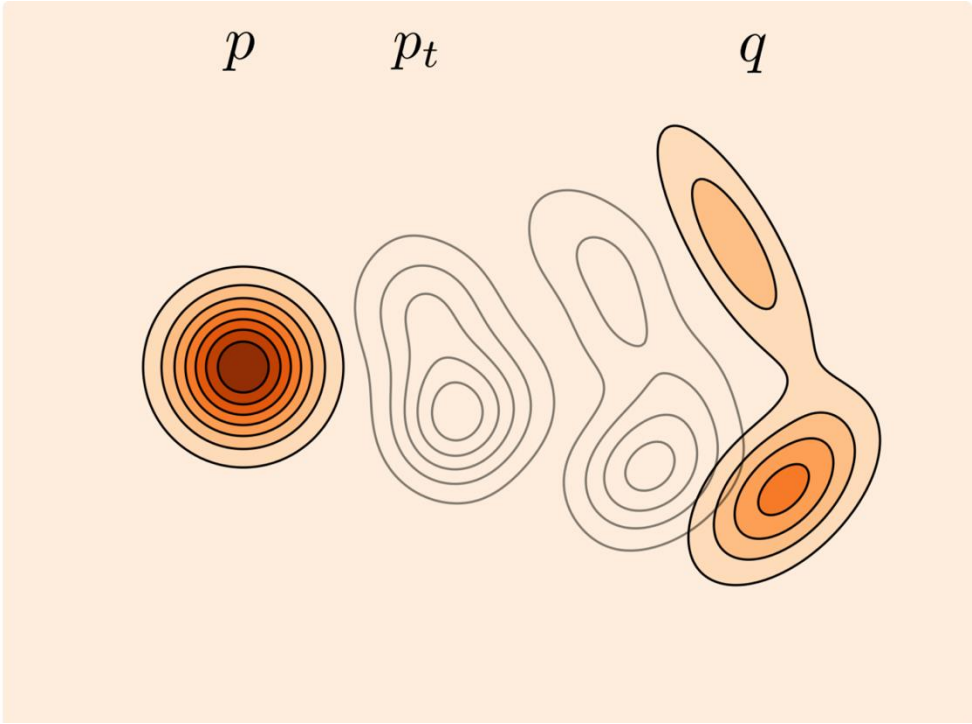


Flow Matching

Gaussian Flow matching

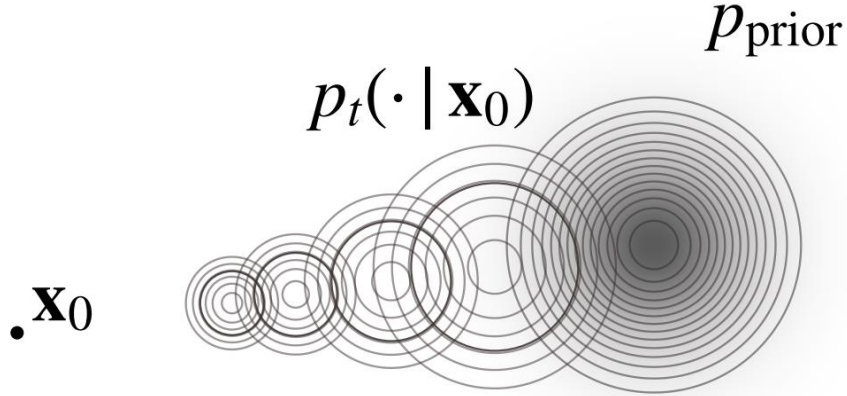
Lipman, Yaron, et al. ICLR 2023

- **Main Idea:** learn a continuous flow to transport samples from one distribution to the other.



Flow matching

- **Conditional Path**



Flow matching

- **Conditional Path**
- **Velocity Field:** find a velocity field that the induced ODE produces marginal distribution of $x(t)$ that matches with p_t at each time t.

$$\frac{\mathrm{d}\mathbf{x}(t)}{\mathrm{d}t} = \mathbf{v}_t(\mathbf{x}(t)), \quad t \in [0,1],$$

$$\frac{\partial p_t(\mathbf{x})}{\partial t} + \nabla \cdot \left(\mathbf{v}_t(\mathbf{x})\, p_t(\mathbf{x})\right) = 0.$$

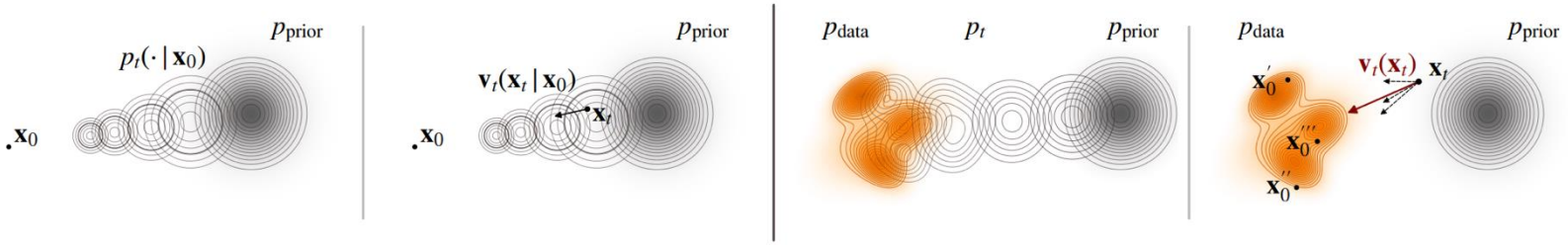
Flow matching

- **Conditional Path**
- **Velocity Field**
- **Learning via Conditional Strategy**

$$\mathcal{L}_{\mathrm{FM}}(\phi) = \underbrace{\mathbb{E}_{t,\mathbf{z}\sim\pi(\mathbf{z}),\,\mathbf{x}_t\sim p_t(\cdot|\mathbf{z})}\left[\left\|\mathbf{v}_\phi(\mathbf{x}_t,t) - \mathbf{v}_t(\mathbf{x}_t \mid \mathbf{z})\right\|^2\right]}_{\mathcal{L}_{\mathrm{CFM}}(\phi)} + C$$

Flow matching

- **Conditional Path**
- **Velocity Field**
- **Learning via Conditional Strategy**



Attention and Transformer

Why Transformers: Going Beyond CNN Models

- Limitations of CNNs:
  - Spatially local processing
  - With increased depth: Lower resolution, Wider (richer) features

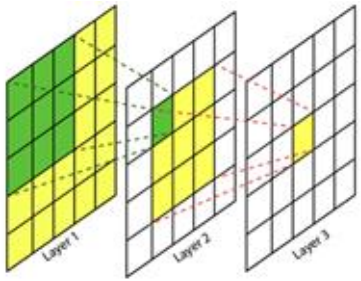
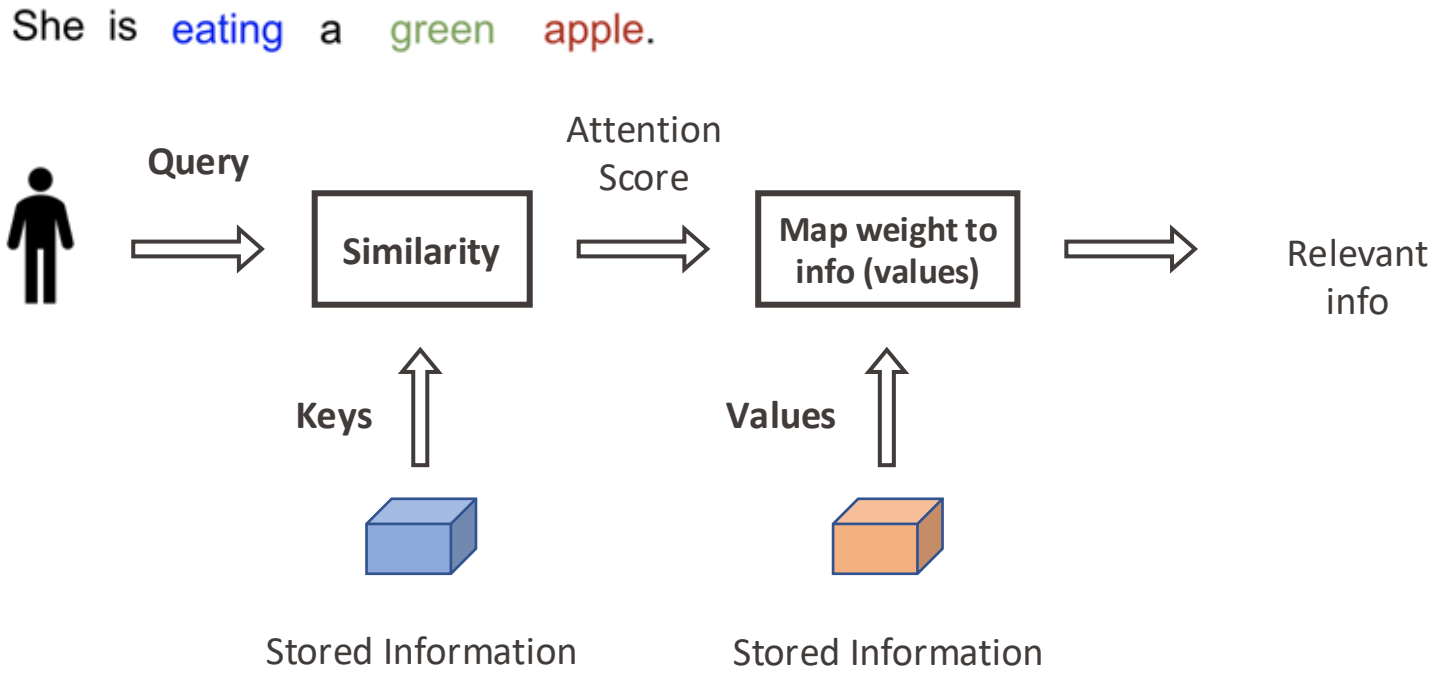


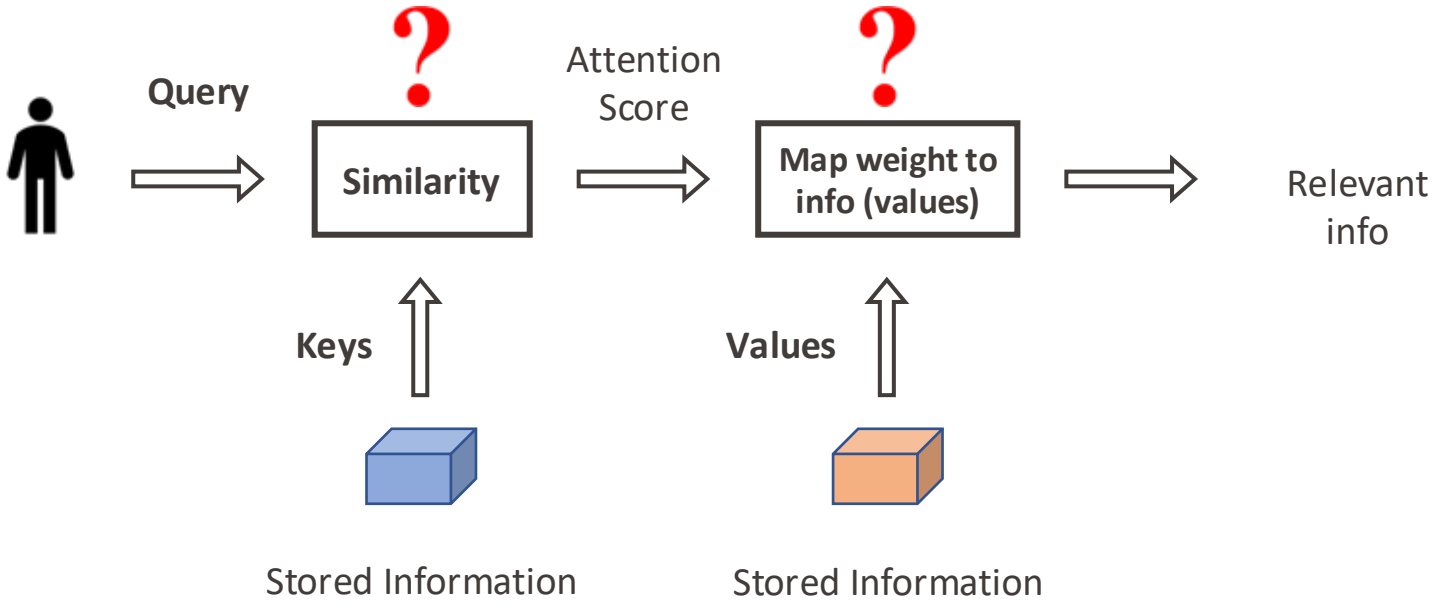
Image by Haoning Lin et al, Remote Sensing 2017

- Desired Capabilities:
  - Long range processing

Understanding Attention

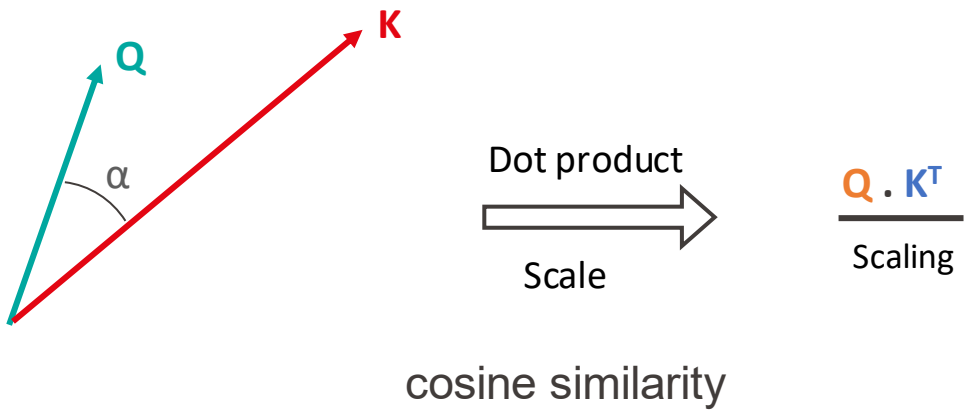


Understanding Attention



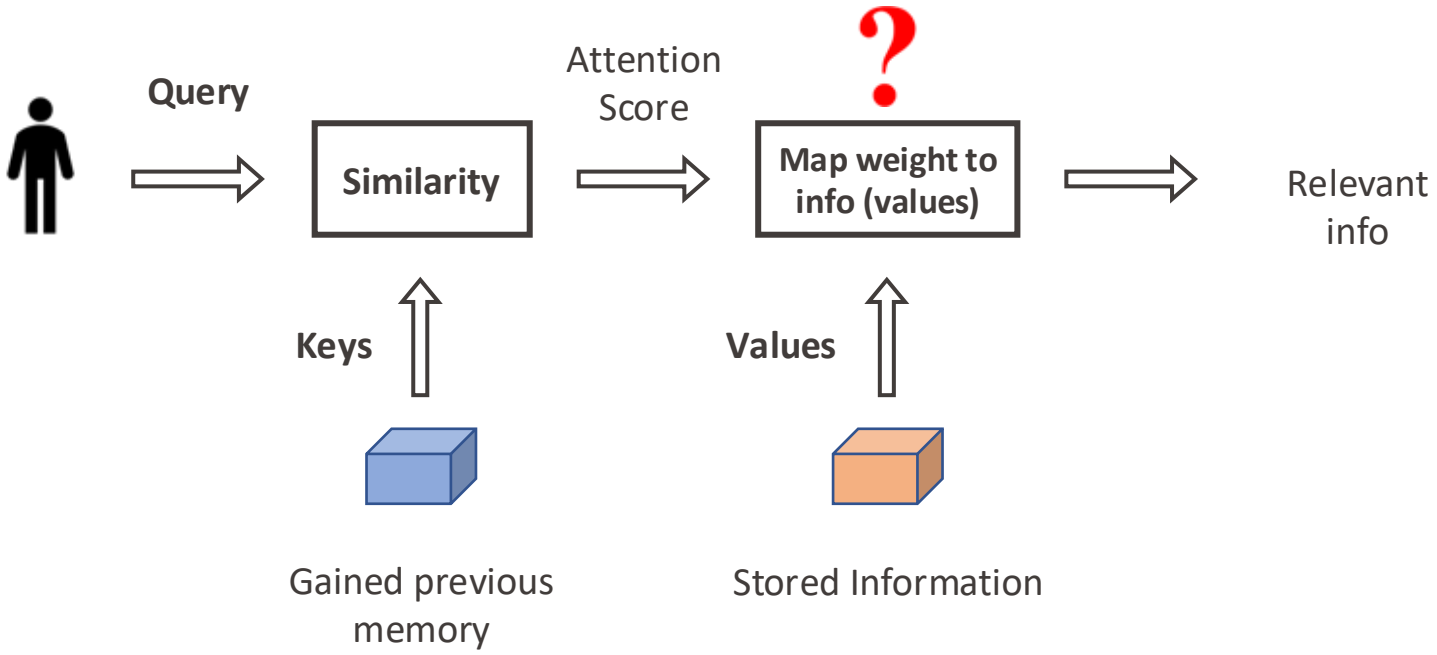
Attention Score

Compute pairwise similarity between each query and key.



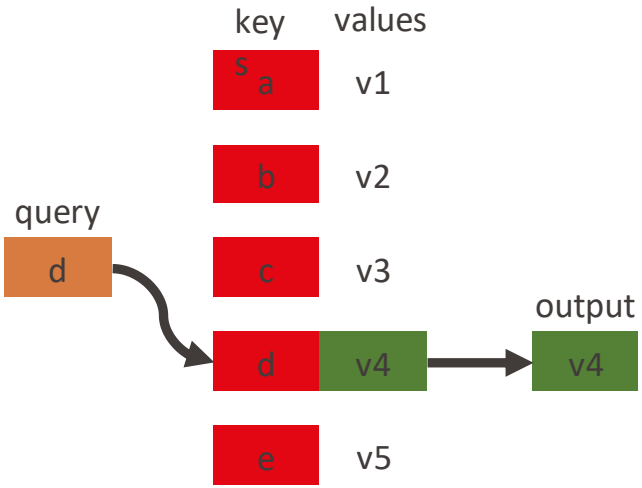
cosine similarity

Understanding Attention

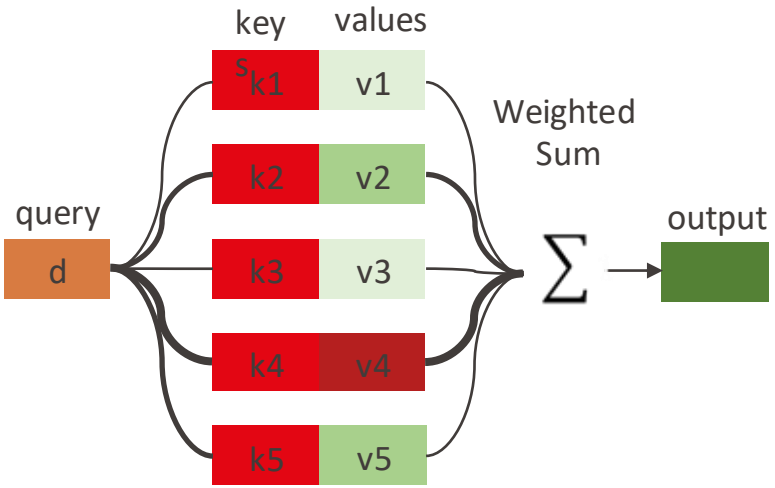


Understanding Attention

For a specific query, we have a table of keys that map to values. The query matches one of the keys, returning its value.



The query matches all keys softly, to a weight between 0 and 1. The keys' values are multiplied by the weights and summed.



output = $\sum_{i=1}^{5} \alpha_i Vi$; α_i are attention weights

Self-attention

Attention: A mechanism to selectively exchange and process information

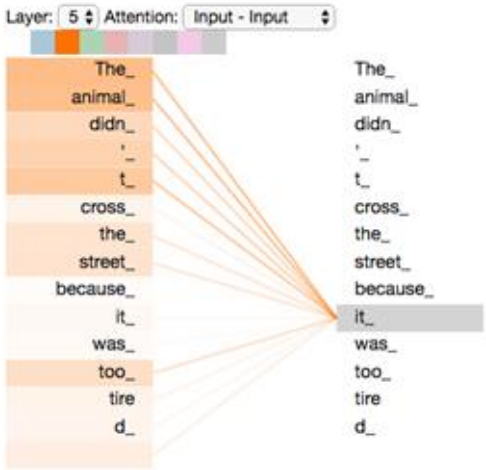
How? By averaging **values** associated to **keys** matching a **query**

Self-attention: Attention from each token to all other tokens

Attention? Attention! (Lilian Weng, 2018)

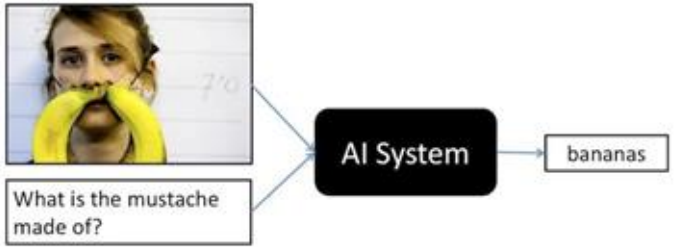
Self-Attention vs Cross-Attention

- Self-Attention: Queries, keys, and values are generated from the same source.



The Illustrated Transformer (Jay Alammar, 2018)

- Cross-Attention: Queries are generated from a source different from keys, and values.



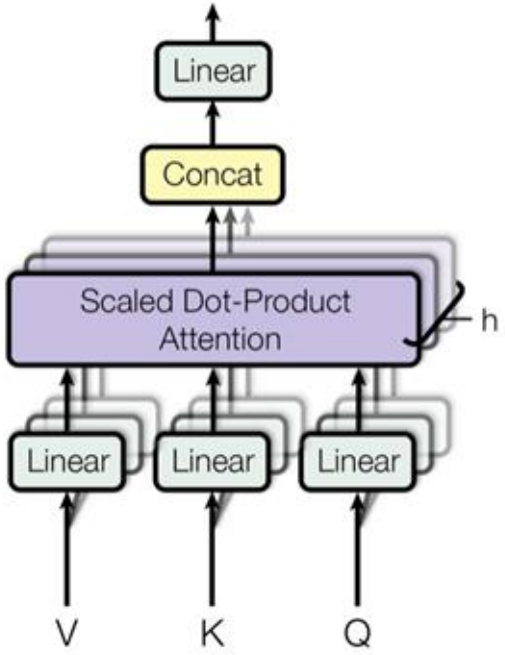
VQA: Visual Question Answering, ICCV 2015

Multi-head attention

Idea: Perform self-attention multiple times in parallel and combine the results!

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_\text{h})W^O$$

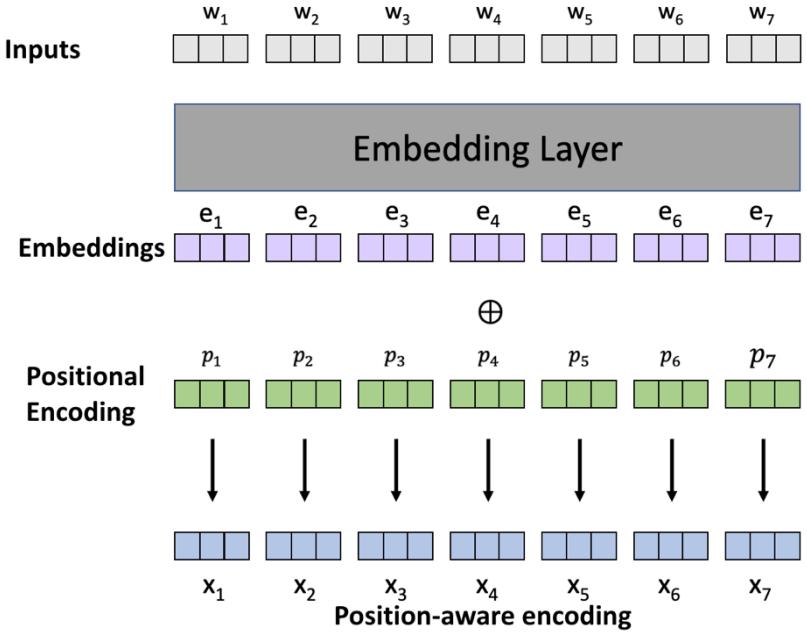
$$\text{where head}_\text{i} = \text{Attention}(XW_i^Q, XW_i^K, XW_i^V)$$

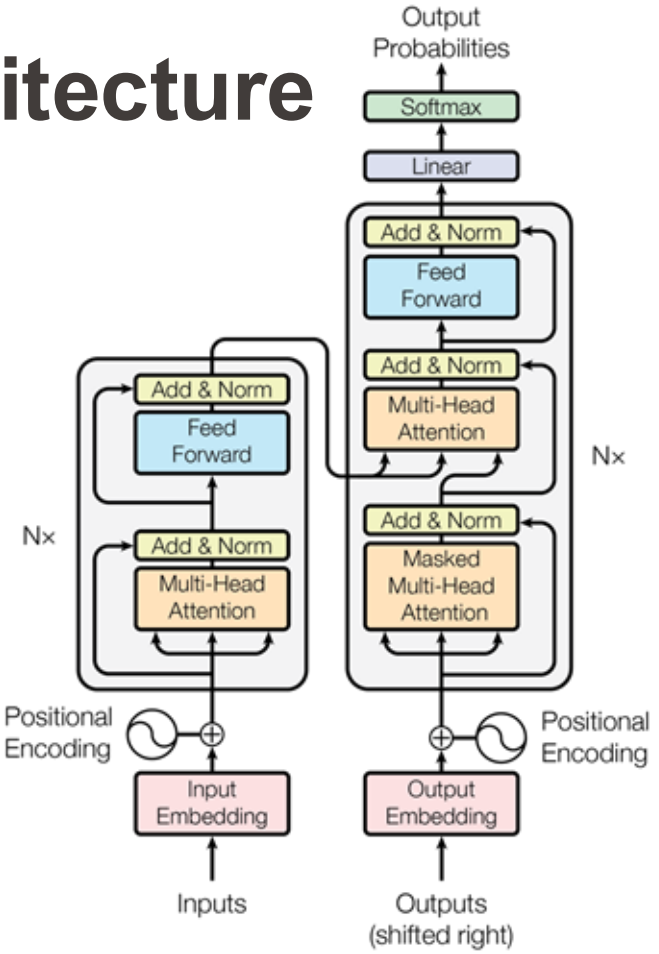


Still easy to vectorize, but the model can now jointly attend to information from different subspaces at different positions

Attention Is All You Need (Vaswani et al., 2017)

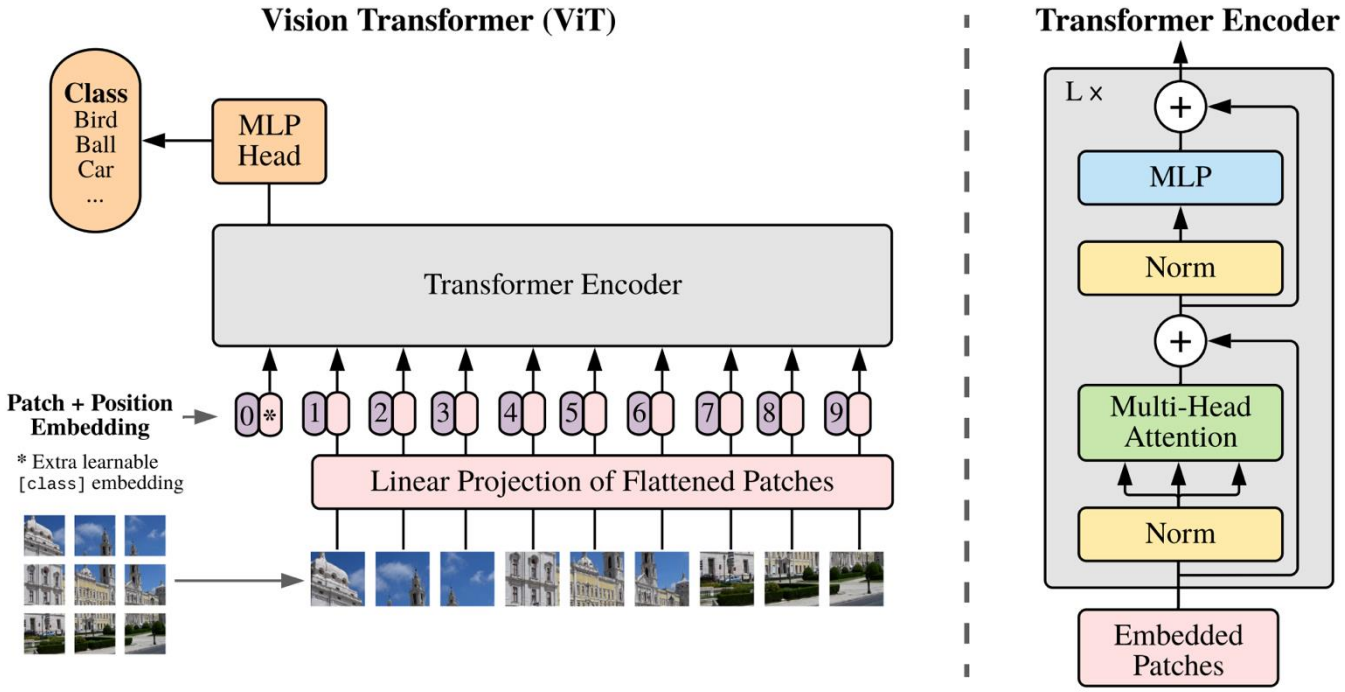
The Transformer architecture





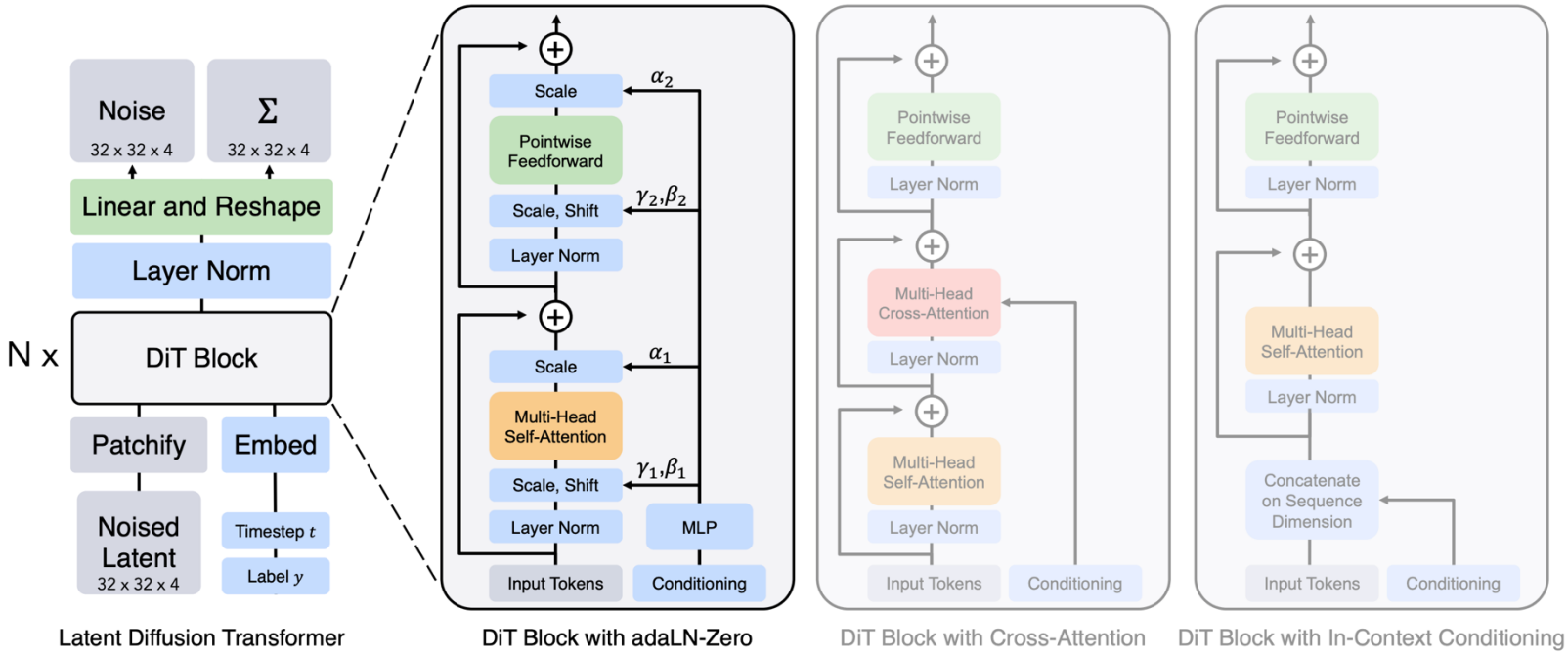
- Attention Is All You Need (Vaswani et al., 2017)

Vision Transformer



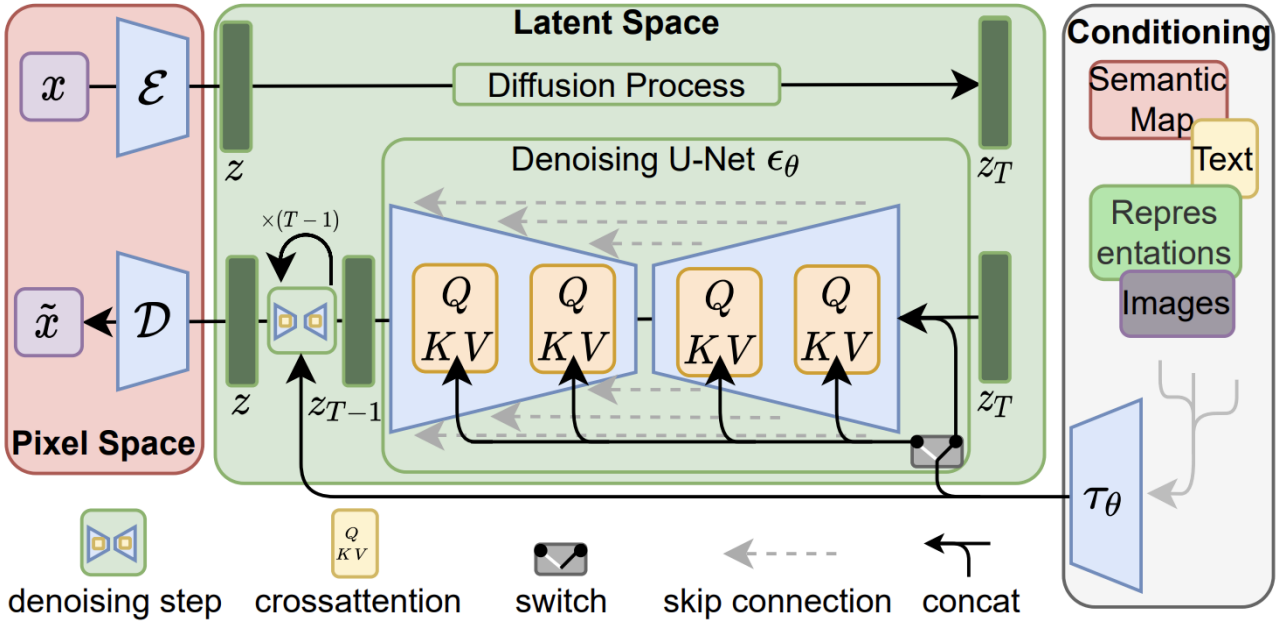
- AN IMAGE IS WORTH 16X16 WORDS

Diffusion Transformer



- Scalable Diffusion Models with Transformers

Latent Diffusion model



- High-Resolution Image Synthesis with Latent Diffusion Models

Controllable Generation

CFG

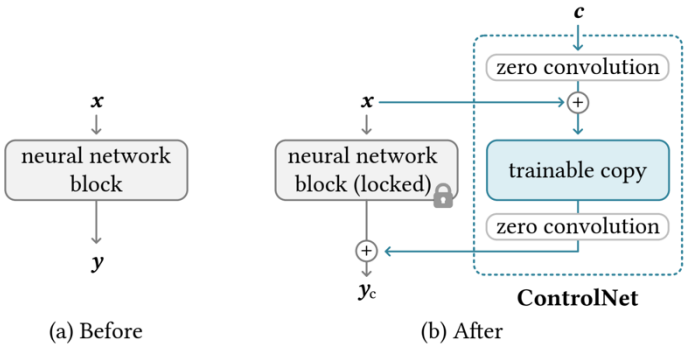
$$\epsilon_\theta(x_t, t, c),\ x_\theta(x_t, t, c),\ s_\theta(x_t, t, c),\ v_\theta(x_t, t, c)$$

- First, when training the model, we add the condition on top of the temporal condition.

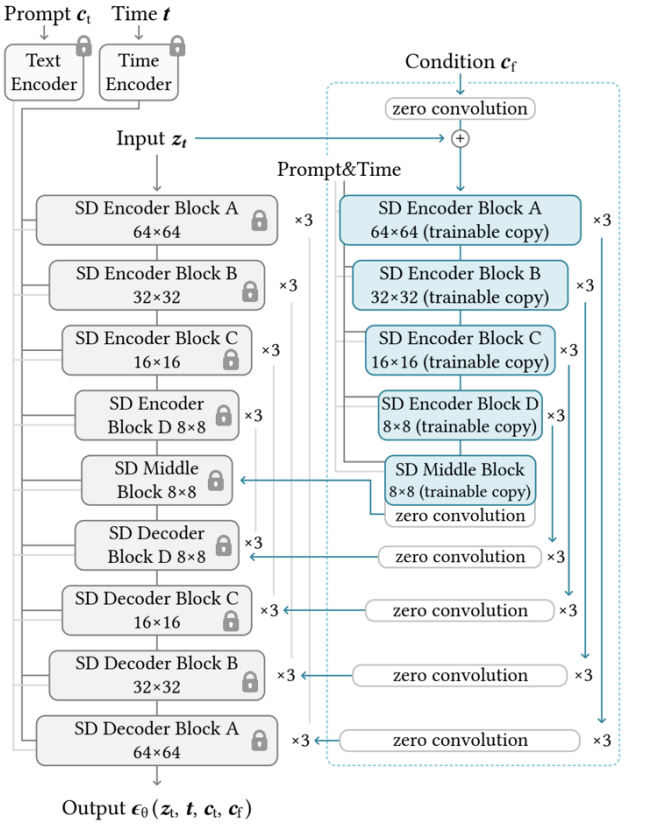
```
predicted_noise = model(x_t, t, c)
```

$$w \cdot \epsilon_\theta(x_t, t, c) + (1 - w) \cdot \epsilon_\theta(x_t, t, \varnothing),$$
where w is the guidance strength.

ControlNet



(a) Before (b) After



(a) Stable Diffusion (b) ControlNet

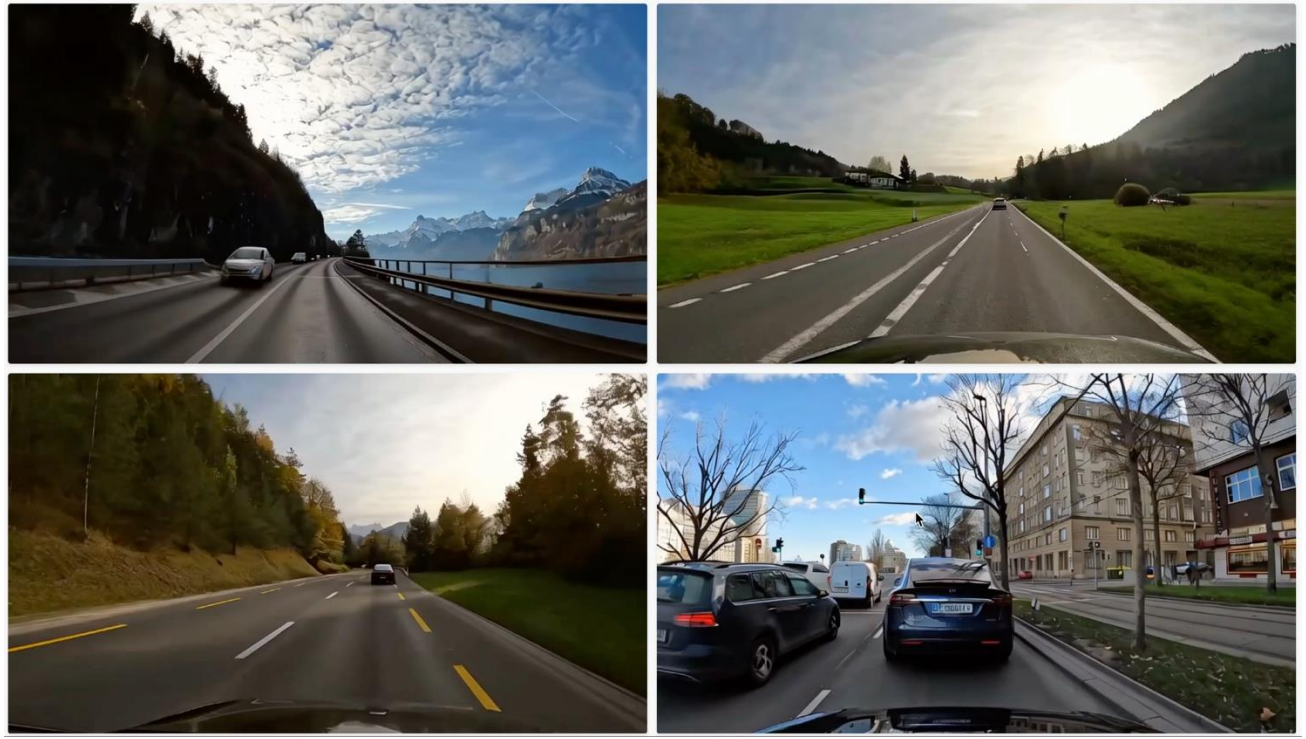
- [Adding Conditional Control to Text-to-Image Diffusion Models]

ControlNet

ControlNet

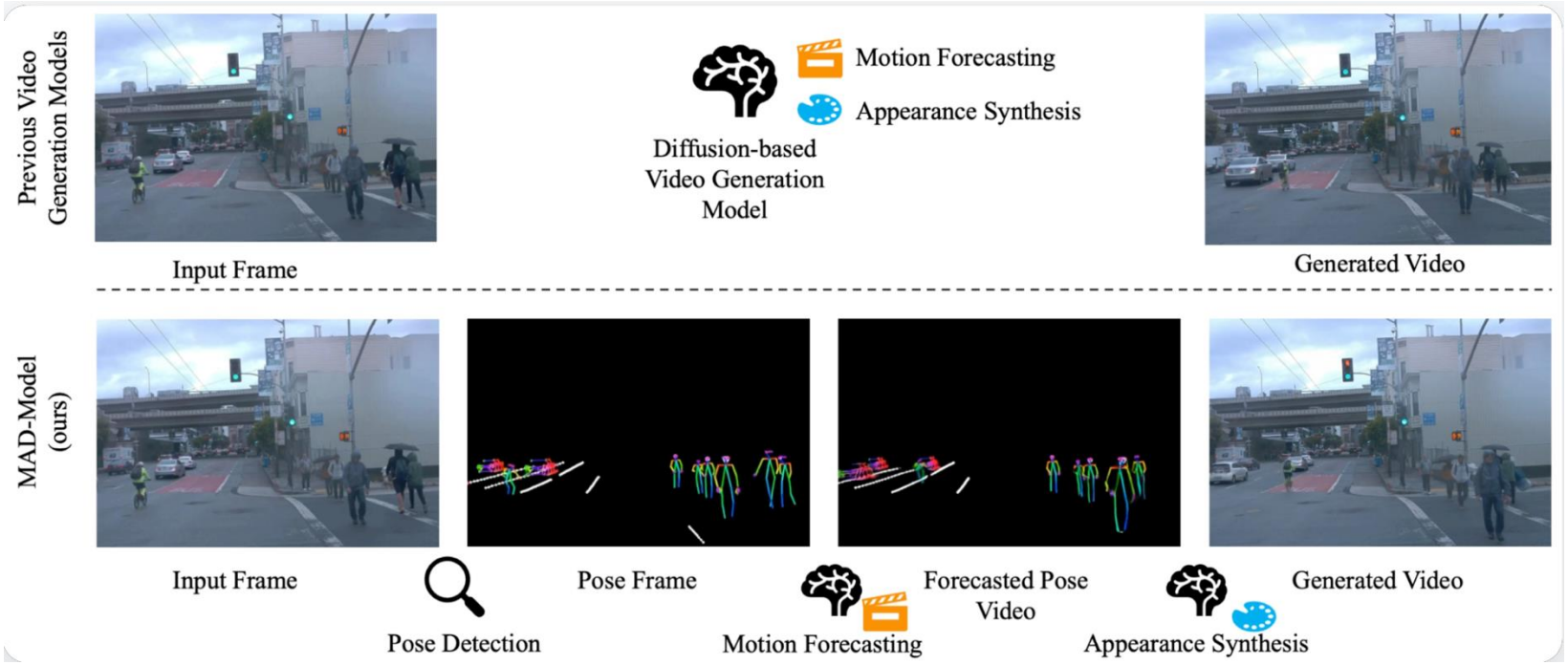
World Modeling

GEM



- GEM: A Generalizable Ego-Vision Multimodal World Model for Fine-Grained Ego-Motion, Object Dynamics, and Scene Composition Control

MAD



MAD: Motion Appearance Decoupling for efficient Driving World Models

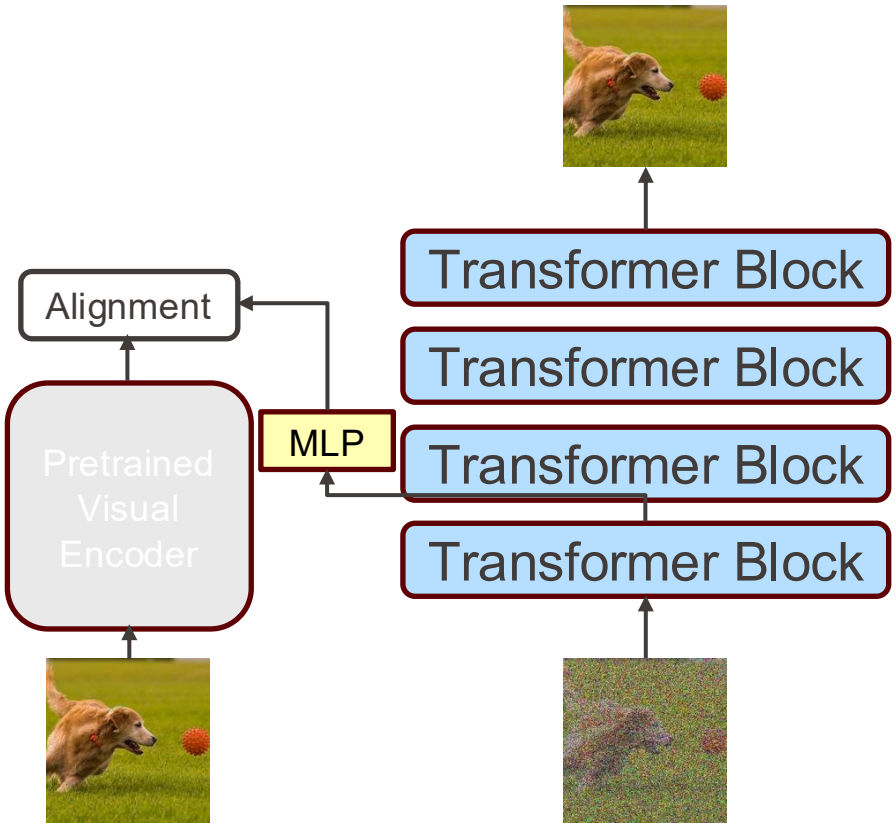
Accelerating the Training

Motivation

Cosmos was trained on **10,000 GPUs** over **3 months.**

Imagen 3 was trained on 1024 **TPUs** for **several months**.

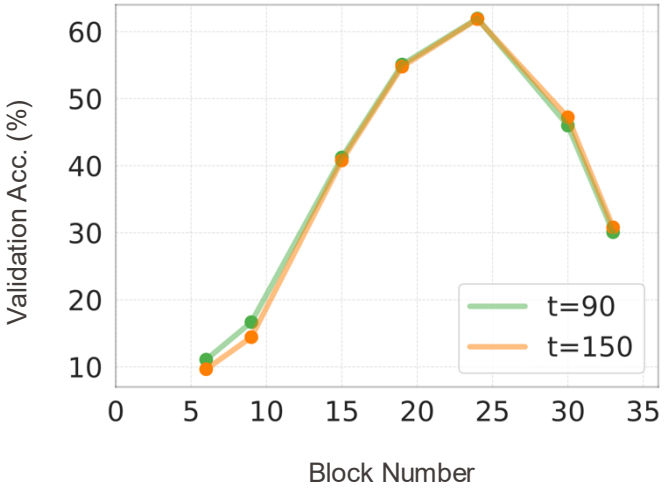
REPA



- Accelerates training
- Extra cost and parameters
- Not easy to extend beyond visual domain

Yu, Sihyun, et al. "Representation Alignment for Generation" ICLR 2025.

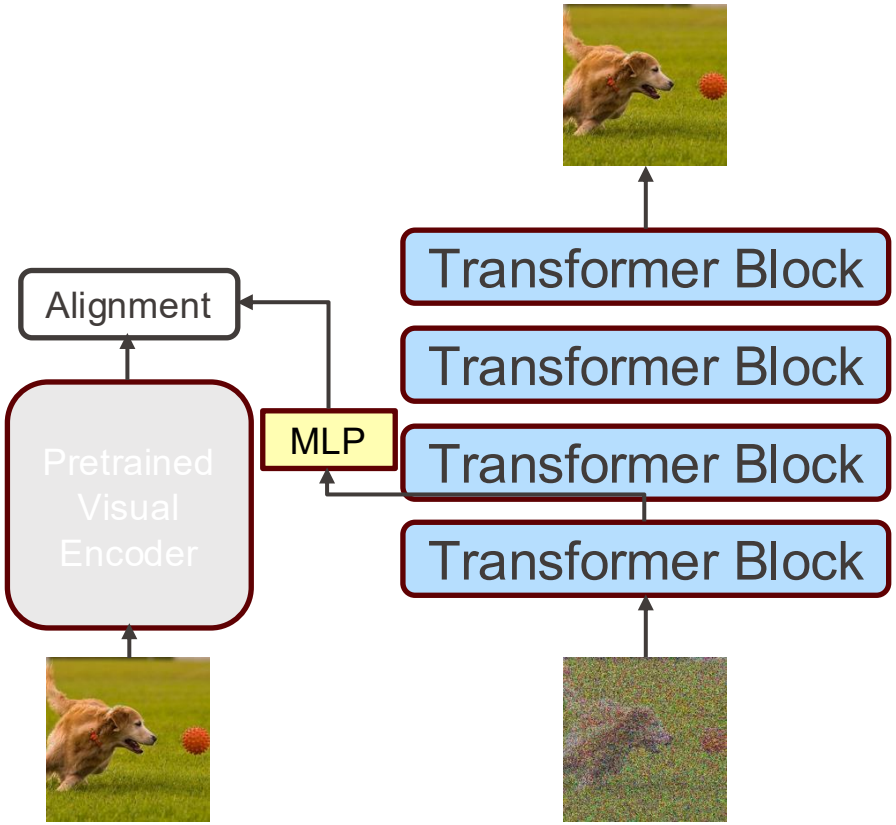
Observation



Linear probing of diffusion models for classification on ImageNet (256×256) shows that the deeper layers learn discriminative features.

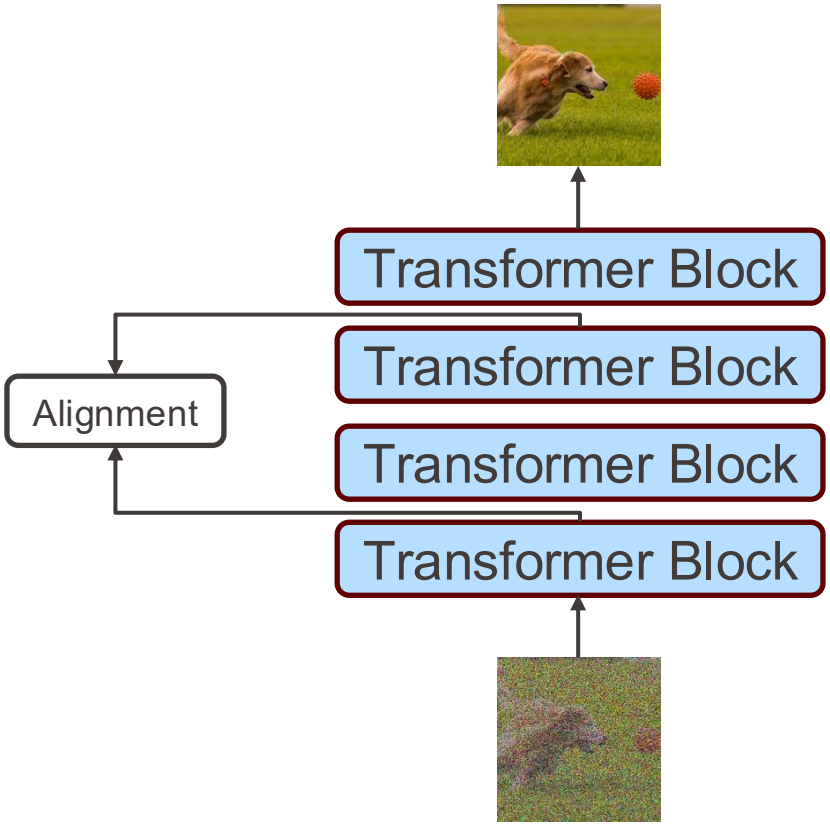
Xiang, Weilai, et al. "Denoising diffusion autoencoders are unified self-supervised learners." ICCV 2023.
Mukhopadhyay, Soumik, et al. "Do text-free diffusion models learn discriminative visual representations?." ECCV 2024.
Stracke, Nick, et al. "Cleandift: Diffusion features without noise." CVPR 2025.

REPA



Yu, Sihyun, et al. "Representation Alignment for Generation" ICLR 2025.

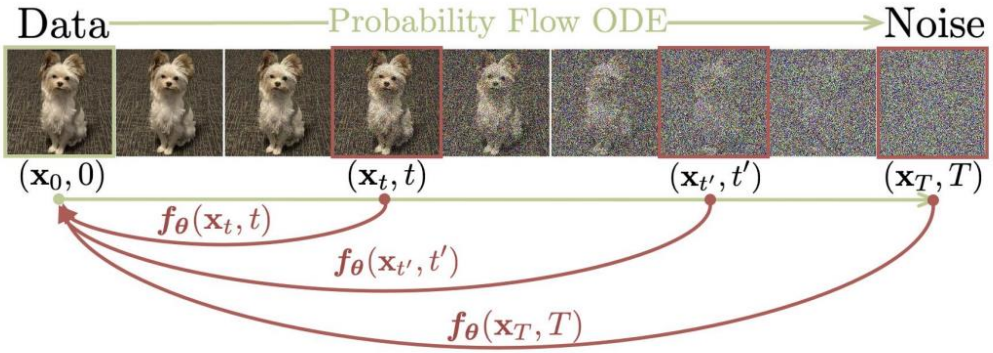
LayerSync



Haghighi, Yasaman, et al. "LayerSync: Self-aligning Intermediate Layers." ICLR 2026.

Accelerating the Inference

Consistency models



Input: dataset $\mathcal{D}$, initial model parameter $\boldsymbol{\theta}$, learning rate η, step schedule $N(\cdot)$, EMA decay rate schedule $\mu(\cdot)$ $d(\cdot,\cdot)$, and $\lambda(\cdot)$

$\boldsymbol{\theta}^- \leftarrow \boldsymbol{\theta}$ and $k \leftarrow 0$

repeat

  Sample $\mathbf{x} \sim \mathcal{D}$, and $n \sim \mathcal{U}[\![1, N(k)-1]\!]$

  Sample $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$

  $\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\theta}^-) \leftarrow$

    $\lambda(t_n) d(\boldsymbol{f}_{\boldsymbol{\theta}}(\mathbf{x} + t_{n+1}\mathbf{z}, t_{n+1}), \boldsymbol{f}_{\boldsymbol{\theta}^-}(\mathbf{x} + t_n \mathbf{z}, t_n))$

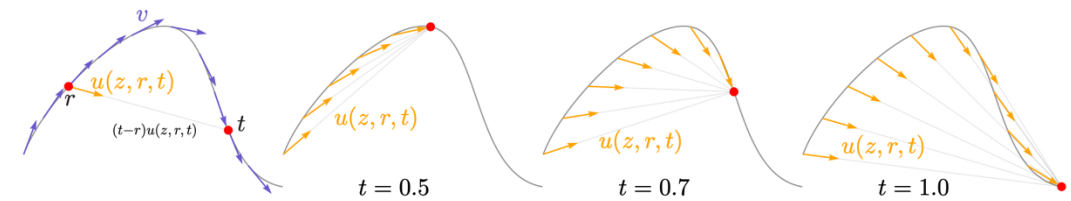
  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\theta}^-)$

  $\boldsymbol{\theta}^- \leftarrow \text{stopgrad}(\mu(k)\boldsymbol{\theta}^- + (1-\mu(k))\boldsymbol{\theta})$

  $k \leftarrow k + 1$

until convergence

MeanFlow



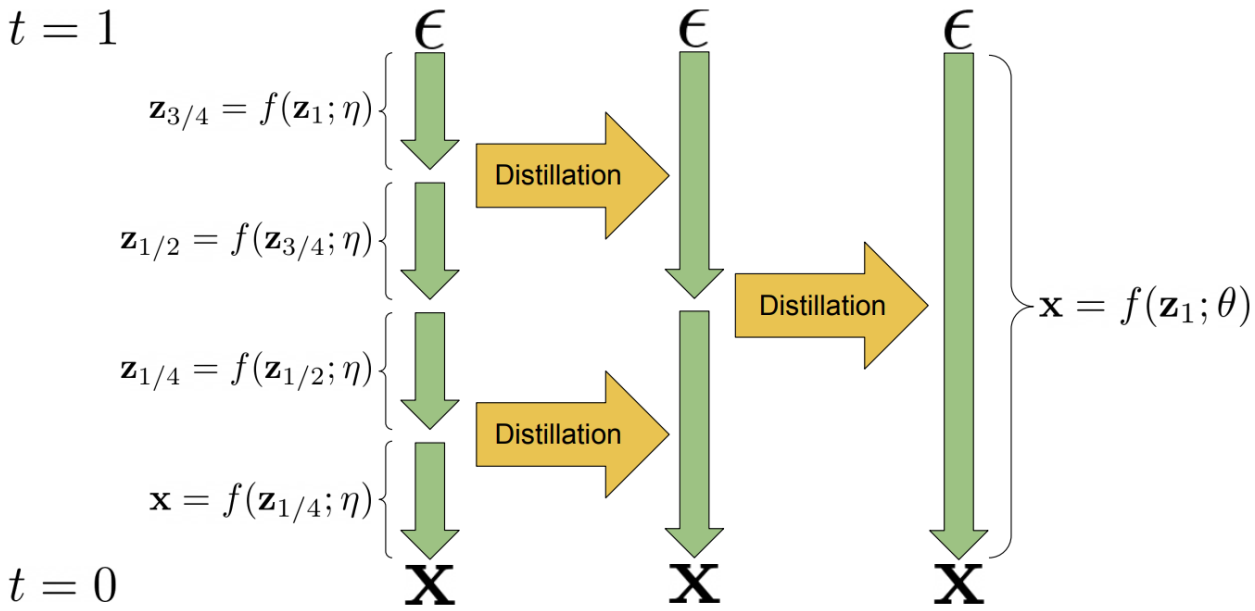
Algorithm 1 MeanFlow: Training.

Note: in PyTorch and JAX, `jvp` returns the function output and JVP.

```
# fn(z, r, t): function to predict u
# x: training batch

t, r = sample_t_r()
e = randn_like(x)

z = (1 - t) * x + t * e
v = e - x

u, dudt = jvp(fn, (z, r, t), (v, 0, 1))

u_tgt = v - (t - r) * dudt
error = u - stopgrad(u_tgt)

loss = metric(error)
```

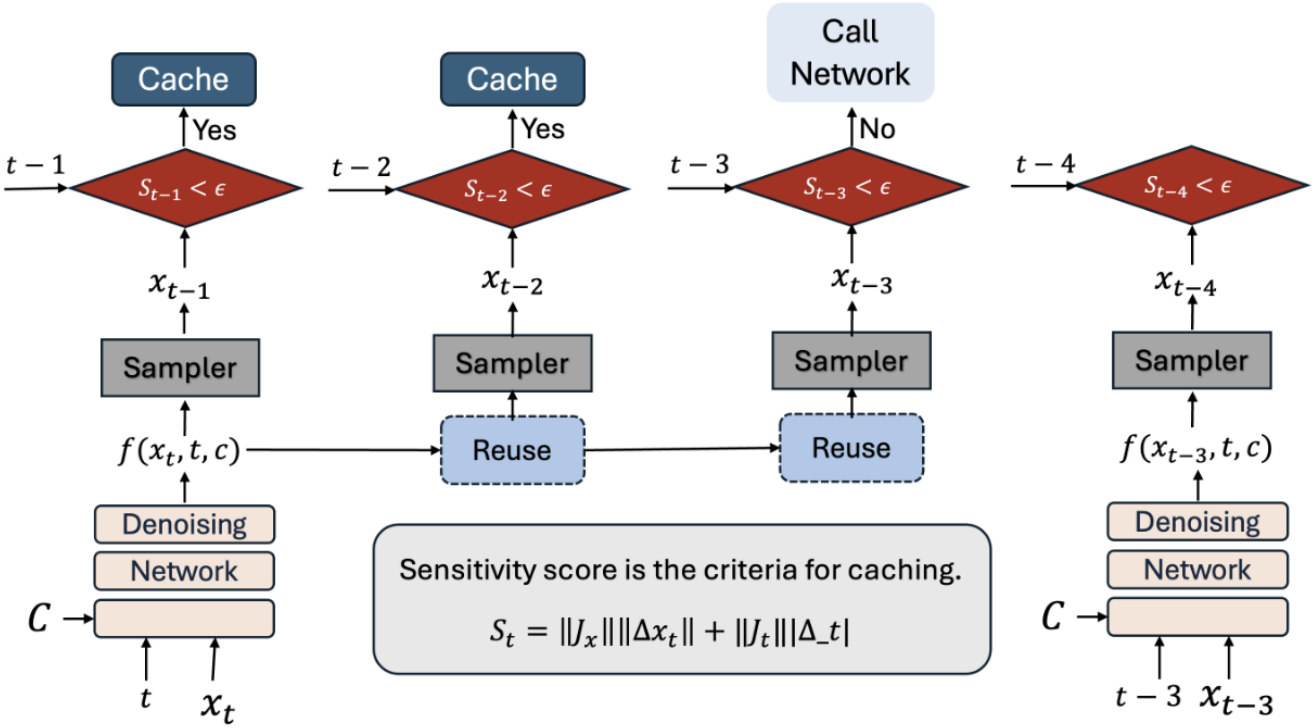
Mean Flows for One-step Generative Modeling

Distillation



PROGRESSIVE DISTILLATION FOR FAST SAMPLING OF DIFFUSION MODELS

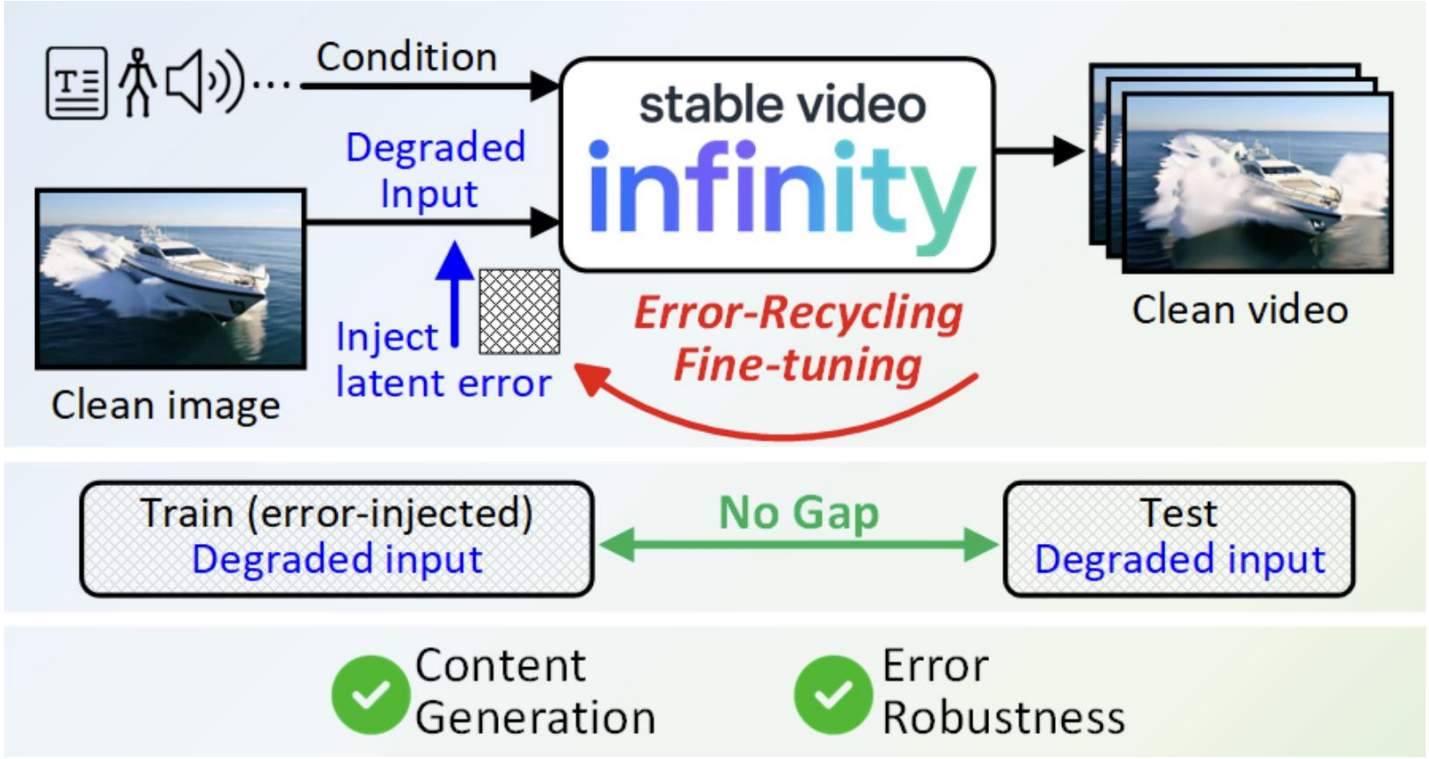
Caching



SenCache: Accelerating Diffusion Model Inference via Sensitivity-Aware Caching

Long Generation

SVI


Condition
Degraded Input
stable video infinity
Clean image
Inject latent error
Error-Recycling Fine-tuning
Clean video
Train (error-injected) Degraded input
No Gap
Test Degraded input
Content Generation
Error Robustness

EPFL VITA

Thank you for your attention